

A Lattice-based Designated Verifier zkSNARK from Standard Assumptions

Mohammad Sadegh Ahmadi, Taraneh Eghlidos, Behzad Abdolmaleki, and Ngoc Khanh Nguyen

Abstract—Designated Verifier zero-knowledge Succinct Non-Interactive Arguments of Knowledge (DV-zkSNARKs) are cryptographic argument systems in which the ability to verify proofs is restricted to a designated verifier. Unlike publicly verifiable zkSNARKs, these constructions ensure that only an authorized party can validate the correctness of the proof. Existing lattice-based DV-zkSNARK constructions typically rely on either linear-only encryption (LOE) or linear targeted malleability (LTM). The former does not guarantee security against quantum adversaries, while the latter restricts knowledge soundness to the non-adaptive setting. To address these limitations, we propose an inner product argument system that relies solely on the hardness of the Module Short Integer Solution (MSIS) assumption and achieves knowledge soundness in the random oracle model. This construction enables a designated verifier, holding a secret key, to succinctly verify inner product of a committed witness with an arbitrary vector. By combining our argument system with a linear probabilistic checkable proof (LPCP) compiler, to the best of our knowledge, we obtain the first DV-zkSNARK construction based on standard assumptions. Our implementation achieves prover and verification times comparable to the state of the art, while reducing public parameter size by a factor of 10, at the cost of a 2.5× increase in proof size.

Index Terms—DV-zkSNARK, Linear targeted malleability, Adaptive soundness, Lattice-based.

I. INTRODUCTION

Succinct Non-Interactive Arguments of Knowledge (SNARKs) are fundamental cryptographic primitives that enable provers to efficiently convince verifiers of the validity of computational statements using proofs significantly smaller than the underlying computations [1]. Over the past decade, SNARKs have seen rapid theoretical advancements and widespread practical adoption, particularly in blockchain technologies and verifiable computation frameworks [2]–[6]. While most SNARK constructions are publicly verifiable, there exist settings in which the prover intends to convince only an authorized verifier. Such schemes, referred to as Designated Verifier SNARKs (DV-zkSNARKs), have been studied in the context of verifiable computation [7], indistinguishability obfuscation [8], and anonymous credentials [9].

The anticipated threat of quantum computing has spurred the development of post-quantum secure SNARKs, with

lattice-based approaches being particularly attractive due to their strong theoretical foundations and presumed resistance to quantum attacks. The first publicly verifiable lattice-based SNARK was proposed by Albrecht *et al.* [10]. However, as this scheme relied on non-standard assumptions, Wee and Wu [11] demonstrated attacks exploiting structural weaknesses in existing lattice-based SNARKs. LaBRADOR [12] is another lattice-based argument system that is publicly verifiable but suffers from linear-time verification. Several works have explored constructing SNARKs by combining polynomial commitment schemes with PIOP compilers [13]–[15], but the resulting large parameter sizes lead to inefficiency. In contrast, DV-SNARKs often achieve better efficiency. Bitansky *et al.* [16] introduced a generic compiler for constructing SNARKs in the preprocessing model, based on the *linear-only encryption* (LOE) assumption. Their framework builds on an LPCP compiler, which provides succinct verification for Rank-1 Constraint System (R1CS) circuits, and employs homomorphic encryption over the encrypted challenges to compute the proof, allowing any verifier holding the secret key to verify it. The LOE assumption guarantees that an adversary with access to encrypted challenges can only form valid linear combinations, ensuring soundness. The first lattice-based SNARK in this framework was introduced by Boneh *et al.* [8], using LWE-based homomorphic encryption to instantiate the LOE property. Subsequent works extended this approach to support stronger guarantees such as zero-knowledge, knowledge soundness, and improved efficiency [17]–[21]. However, recent results by Debris *et al.* [22] demonstrated that the LOE assumption fails against quantum adversaries, undermining the post-quantum security of these constructions. To address this, Steinfeld *et al.* [23] proposed replacing LOE with a weaker assumption called *linear targeted malleability* (LTM), which underlies the current state-of-the-art lattice-based DV-SNARK. Nevertheless, this assumption is not yet well established. Moreover, Bitansky *et al.* [16] showed that constructions based on the LTM assumption cannot achieve adaptive soundness, requiring the prover to commit to the witness before observing the CRS which is strictly weaker security notion.

A. Our Contribution

Given this context, the open challenge is to devise lattice-based SNARKs that simultaneously offer post-quantum security, relying on established lattice assumptions, while maintaining efficient verification processes.

We propose an inner product argument based on a modified version of the Ajtai commitment scheme [24]. Our protocol is LPCP-friendly, enabling efficient verification of inner product

Mohammad Sadegh Ahmadi is with Department of Electrical Engineering, Sharif University of Technology, Tehran, Iran (e-mail: mohammad-sadegh.ahmadi@ee.sharif.edu).

Taraneh Eghlidos is with the Electronics Research Institute, Sharif University of Technology, Tehran, Iran (e-mail: teghlidos@sharif.edu).

Behzad Abdolmaleki is with the Department of Computer Science, University of Sheffield, Sheffield, UK (e-mail: behzad.abdolmaleki@sheffield.ac.uk).

Ngoc Khanh Nguyen is with the Department of Informatics, King's College London, London, UK (e-mail: ngoc_khanh.nguyen@kcl.ac.uk).

relations between committed vectors and challenge vectors. By integrating this protocol with the LPCP compiler, we construct an efficient DV-zkSNARK that is both succinct and secure under the hardness of the Module-SIS assumption. In contrast to prior DV-zkSNARK constructions, our protocol is not based on the LTM assumption and additionally satisfies the knowledge soundness property in the random oracle model. As a result, it achieves *adaptive soundness*, ensuring security even when the prover's statement is chosen after the system parameters have been generated. To the best of our knowledge, this is the first lattice-based DV-zkSNARK based solely on standard assumptions. We also provide a proof-of-concept implementation of our construction, allowing for a thorough and precise evaluation of its practical performance. Our experimental results demonstrate the feasibility of the design in real-world scenarios. The source code is publicly available on GitHub¹.

B. Technical Overview

We provide a high-level overview of our construction and the techniques underlying our approach.

A Lattice-based Inner-Product Argument: We introduce a type of inner-product argument in which the prover first commits to a witness vector $\vec{\mathbf{b}}$ along with a noise vector $\vec{\mu}$ using a revised version of the Ajtai commitment scheme. The verifier then samples and sends a uniformly random challenge vector $\vec{\mathbf{u}}_{\text{chall}}$. The prover computes the inner product of the witness with this challenge and provides a corresponding proof. In other words, our goal is to construct an argument system for the following relation:

$$\mathbb{R}_{\text{IPA}} = \left\{ \begin{array}{l} \text{pp} : (\mathbf{A}_{\text{com}}, \mathbf{A}_{\text{hide}}) \\ \mathbb{X} : (\vec{\mathbf{c}}, \vec{\mathbf{u}}_{\text{chall}}, \mathbf{h}) \\ \mathbb{W} : (\vec{\mathbf{b}}, \vec{\mu}) \end{array} : \begin{array}{l} \vec{\mathbf{c}} = \mathbf{A}_{\text{com}} \vec{\mathbf{b}} + \mathbf{A}_{\text{hide}} \vec{\mu} \\ \mathbf{h} = \langle \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}, \vec{\mathbf{u}}_{\text{chall}} \rangle \\ \left\| \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} \right\|_{\infty} \leq \beta_{\text{open}} \end{array} \right\}. \quad (1)$$

We carefully modify the commitment matrices $\mathbf{A}_{\text{com}}$ and $\mathbf{A}_{\text{hide}}$, constructing them using submatrices that possess trapdoors (see Section III). This modification enables a verifier holding these trapdoors, along with a low-norm secret key $\vec{\mathbf{p}}$, to compute a trapdoor-dependent vector $\vec{\mathbf{z}}_G$ for any uniformly sampled challenge $\vec{\mathbf{u}}_{\text{chall}}$. This procedure is described in the ChallTrap algorithm (Figure 4). As shown in Claim 4.1, the inner product of the witness with $\vec{\mathbf{u}}_{\text{chall}}$ reduces to verifying the following relation:

$$\mathbf{h} = \langle \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}, \vec{\mathbf{u}}_{\text{chall}} \rangle = \langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle + \langle \vec{\mathbf{z}}_G, \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} \rangle. \quad (2)$$

The verifier can compute $\langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle$, as they know both $\vec{\mathbf{p}}$ and $\vec{\mathbf{c}}$. However, they still need $\langle \vec{\mathbf{z}}_G, \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} \rangle$. A naive approach would be for the verifier to send $\vec{\mathbf{z}}_G$ to the prover, who returns $\mathbf{h}' = \langle \vec{\mathbf{z}}_G, \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} \rangle$. The verifier could then check if $\mathbf{h} = \langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle + \mathbf{h}'$. However, a malicious prover could add a fixed offset $\mathbf{h}''$ to their claimed responses for both challenges and still pass verification, thereby breaking soundness.

To overcome this, we adopt a masking technique: embedding $\vec{\mathbf{z}}_G$ into a statistically uniform-looking vector using the Gaussian2Uniform algorithm (Figure 5). This algorithm takes $\vec{\mathbf{u}}_{\text{chall}}$, $\vec{\mathbf{z}}_G$, and a Gaussian-sampled secret $\mathbf{z}_{\text{sec}}$ (matching the distribution of $\vec{\mathbf{z}}_G$) to produce $\vec{\mathbf{u}}_{\text{z,chall}} = \vec{\mathbf{z}}_G + \mathbf{z}_{\text{sec}} \vec{\mathbf{u}}_{\text{chall}}$. If the conditions of Lemma 2.4 hold, this vector is statistically close to uniform. Then, the prover's response to this challenge is $\mathbf{h}_z = \langle \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}, \vec{\mathbf{z}}_G \rangle + \mathbf{z}_{\text{sec}} \mathbf{h}$. Adding $\langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle$ to both sides yields:

$$\mathbf{h}_z + \langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle = (\mathbf{z}_{\text{sec}} + 1) \mathbf{h}. \quad (3)$$

The verifier can now easily check whether Equation 3 holds. Importantly, the prover cannot cheat in this process, as they do not know the value of $\mathbf{z}_{\text{sec}}$.

Extending to Arbitrary Challenges: A verifier can also verify the inner product with an arbitrary vector $\vec{\mathbf{u}}_{\chi}$. Using ChallTrap and Gaussian2Uniform with a new secret $\mathbf{z}_{\text{sec},\chi}$, the verifier constructs a uniform-looking challenge:

$$\vec{\mathbf{u}}_{\text{chall},\chi} = \vec{\mathbf{z}}_{G,\chi} + \mathbf{z}_{\text{sec},\chi} \vec{\mathbf{u}}_{\text{chall}}.$$

The procedure for computing such a challenge is illustrated in Figure 1. The prover's response to this challenge is:

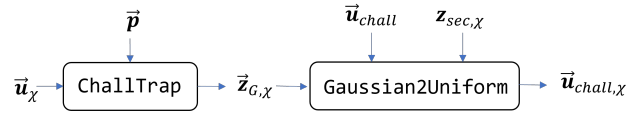


Fig. 1. Converting a desired challenge vector to a uniform one.

$$\mathbf{h}_{\chi} = \langle \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}, \vec{\mathbf{u}}_{\text{chall},\chi} \rangle = \mathbf{z}_{\text{sec},\chi} \mathbf{h} + \langle \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}, \vec{\mathbf{z}}_{G,\chi} \rangle.$$

Subtracting $\mathbf{z}_{\text{sec},\chi} \mathbf{h}$ from $\mathbf{h}_{\chi}$ and adding $\langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle$ to it recovers $\langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle + \langle \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}, \vec{\mathbf{z}}_{G,\chi} \rangle$, which is equal to $\langle \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}, \vec{\mathbf{u}}_{\chi} \rangle$ according to Equation 2.

Hence, the verifier could ask about inner product of any desired vector. Note that the verifier also need to verify that the inner product of witness with $\vec{\mathbf{u}}_{\text{chall},\chi}$ is computed correctly. They can adopt the same approach which they have used for verifying the inner product of witness with $\vec{\mathbf{u}}_{\text{chall}}$ since the $\vec{\mathbf{u}}_{\text{chall},\chi}$ is also a uniform challenge.

II. PRELIMINARIES

A. Notation

We denote the security parameter by λ . For $a, b \in \mathbb{N}$ with $a < b$, write $[a, b] := \{a, a+1, \dots, b\}$, $[a] = [1, a]$. Let d be a power of two. We define the ring of integers of the $2d$ -th cyclotomic field as $\mathcal{R} = \mathbb{Z}[X]/(X^d + 1)$, and the corresponding residue ring modulo q as $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^d + 1)$. For polynomials in $\mathcal{R}$ or $\mathcal{R}_q$, we use lowercase bold letters to represent them, e.g., $\mathbf{f}$. These polynomials are often viewed as d -dimensional vectors over $\mathbb{Z}$ or $\mathbb{Z}_q$, where the vector components correspond to the coefficients. Vectors over $\mathcal{R}$ or $\mathcal{R}_q$ are denoted using bold lowercase letters and arrows (such as $\vec{\mathbf{f}}$), while matrices over these rings are denoted by bold uppercase letters. Vectors over fields like $\mathbb{Z}_q$, for prime q , are denoted using lower case letters and arrows (such as $\vec{f}$).

¹<https://github.com/crypolover1998/LatticeBased-Dv-zkSNARK>

If $\vec{f} \in \mathcal{R}_q^n$ is a vector, then we show the subvector of $\vec{f}$ using $[\vec{f}]_i^j$ which means subvector started from index i to index j . For an element $\mathbf{w} = w_0 + w_1X + \dots + w_{d-1}X^{d-1} \in \mathcal{R}_q$, l_p denotes the p -norm of an element for $p = 1, 2, \infty$. We can represent the coefficients of an element $\mathbf{w}$ in $\mathcal{R}_q$ as a vector in $\mathbb{Z}_q^d$ with $\text{cf}(\mathbf{w}) = [w_0, w_1, \dots, w_{d-1}]^T$. We also define the matrix $\text{Rot}(\mathbf{w})$ as follows:

$$\text{Rot}(\mathbf{w}) = [\text{cf}(\mathbf{w}) | \text{cf}(X\mathbf{w}) | \dots | \text{cf}(X^{d-1}\mathbf{w})] \in \mathbb{Z}_q^{d \times d}.$$

It is easy to check that for $\mathbf{w}_1, \mathbf{w}_2 \in \mathcal{R}_q$, $\mathbf{w}_1 \cdot \mathbf{w}_2 = \text{Rot}(\mathbf{w}_1) \cdot \text{cf}(\mathbf{w}_2)$.

Lemma 2.1. [25] *Let $q = 5 \pmod{8}$ be prime and $m \geq n \geq 1$. Then, for a uniformly random matrix $\mathbf{A} \in \mathcal{R}_q^{n \times m}$, the probability that $\mathbf{A}$ is not full-rank is negligible.*

Lemma 2.2 (Generalized Schwartz-Zippel). [26] *Let $f \in \mathcal{R}_q^{\leq d}[X_1, X_2, \dots, X_\mu]$ be a μ -variate nonzero polynomial over a ring $\mathcal{R}_q$ with per-variable degree at most d . Let $\mathcal{C} \subset \mathcal{R}_q$ be a sampling set. Then we have:*

$$\Pr[f(\vec{r}) = 0 \mid \vec{r} \leftarrow \mathcal{C}^\mu] = \frac{d\mu}{|\mathcal{C}|}$$

Lemma 2.3. [27] *Let $\mathbf{M}$ be any matrix in $\mathbb{Z}_p^{d \times k}$ and let $\mathbf{C}$ be uniformly sampled from $\{0, 1\}^{k \times \lambda}$, and $\|\mathbf{M}\|_\infty = \max_{i,j} |m_{i,j}|$. Then*

$$\Pr\left[\|\mathbf{M} \cdot \mathbf{C} \bmod p\|_\infty < \frac{1}{2} \cdot \|\mathbf{M}\|_\infty\right] \leq 2^{-\lambda}.$$

Lemma 2.4 (Regularity Lemma). [28] *Let $\mathcal{R}_q$ be cyclotomic ring of degree d and $q \geq 2$ be an integer. For positive integer $k \leq l \leq \text{poly}(d)$, Let $\mathbf{A} = [\mathbf{I}_k | \bar{\mathbf{A}}] \in \mathcal{R}_q^{k \times l}$, where $\mathbf{I}_k$ is the identity matrix and $\bar{\mathbf{A}} \in \mathcal{R}_q^{k \times (l-k)}$ is uniformly random. The distribution of $\mathbf{A}\vec{x}$, where each coordinate of $\vec{x} \in \mathcal{R}_q^l$ is chosen from a discrete Gaussian distribution of parameter $\sigma > 2d \cdot q^{\frac{k}{l} + \frac{1}{d}}$, is statistically close to uniform distribution.*

Lemma 2.5 (Modified Regularity Lemma). [29] *Let $q = 5 \pmod{8}$ be prime and $m \geq n$. Take $\sigma > 2d \cdot q^{\frac{n}{m} + \frac{2}{dm}}$. Then the following distributions are statistically close:*

$$\begin{aligned} & \{(\mathbf{A}, \mathbf{A}\vec{x}) \mid \mathbf{A} \leftarrow \mathcal{U}(\mathcal{R}_q^{n \times m}), \vec{x} \leftarrow \mathcal{D}_\sigma^m\} \\ & \{(\mathbf{A}, \vec{u}) \mid \mathbf{A} \leftarrow \mathcal{U}(\mathcal{R}_q^{n \times m}), \vec{u} \leftarrow \mathcal{U}(\mathcal{R}_q^n)\}. \end{aligned}$$

B. Lattices

A subset $\Lambda \subseteq \mathbb{R}^m$ is called a **lattice** if: (i) $\mathbf{0} \in \Lambda$, and for any $\mathbf{x}, \mathbf{y} \in \Lambda$, we have $\mathbf{x} + \mathbf{y} \in \Lambda$; (ii) for each $\mathbf{x} \in \Lambda$, there exists an $\epsilon > 0$ such that the set $\{\mathbf{y} \in \mathbb{R}^m : \|\mathbf{x} - \mathbf{y}\| < \epsilon\} \cap \Lambda = \{\mathbf{x}\}$. We say that $\mathbf{B} \in \mathbb{R}^{n \times m}$ is a basis for Λ if its columns are linearly independent, and define the lattice as $\Lambda = \mathcal{L}(\mathbf{B}) := \{\mathbf{B} \cdot \mathbf{z} : \mathbf{z} \in \mathbb{Z}^m\}$. The span (as a vector space) of the basis of a lattice is the span of the lattice, denoted as $\text{Span}(\Lambda)$. We also define the **dual lattice** as $\Lambda^* = \{\mathbf{w} \in \text{Span}(\Lambda) : \langle \mathbf{w}, \mathbf{v} \rangle \in \mathbb{Z}\}$. For $\mathbf{A} \in \mathcal{R}_q^{n \times m}$ and $\vec{w} \in \mathcal{R}_q^n$, we define the m -dimensional q -ary module lattice $\Lambda_q^\perp(\mathbf{A})$ as follows:

$$\Lambda_q^\perp(\mathbf{A}) = \{\vec{x} \in \mathcal{R}_q^m : \mathbf{A}\vec{x} = \mathbf{0} \pmod{q}\}.$$

C. Discrete Gaussian Distribution

Let $\sigma > 0$ be a parameter and Λ be an m -dimensional lattice. The discrete Gaussian distribution $\mathcal{D}_{\Lambda, \sigma, \mathbf{c}}$ over a lattice coset $\mathbf{c} + \Lambda$ is defined as follows:

$$\rho_{\sigma, \mathbf{c}}(\mathbf{z}) := \exp\left(-\pi \frac{\|\mathbf{z} - \mathbf{c}\|^2}{\sigma^2}\right),$$

$$\mathcal{D}_{\Lambda, \sigma, \mathbf{c}}(\mathbf{z}) := \frac{\rho_{\sigma, \mathbf{c}}(\mathbf{z})}{\sum_{\mathbf{x} \in \Lambda} \rho_{\sigma, \mathbf{c}}(\mathbf{x})}.$$

When $\mathbf{c} = \mathbf{0}$ or $\Lambda = \mathbb{Z}^m$, we omit either from the notation. The smoothing parameter $\eta_\epsilon(\Lambda)$ of a lattice is the smallest $s > 0$ such that $\rho_{1/s}(\Lambda^*) \leq 1 + \epsilon$.

Definition 2.1 (Module-SIS). *Let $q = q(\lambda)$, $n = n(\lambda)$, $m = m(\lambda)$ and $\beta = \beta(\lambda)$. The $\text{MSIS}_{n, m, q, \beta}$ assumption holds if for any PPT adversary $\mathcal{A}$, the following probability is negligible:*

$$\Pr[\mathbf{A}\vec{z} = \mathbf{0} \wedge 0 < \|\vec{z}\| \leq \beta \mid \mathbf{A} \leftarrow \mathcal{U}(\mathcal{R}_q^{n \times m}), \vec{z} \leftarrow \mathcal{A}(\mathbf{A})].$$

D. Commitment Scheme

We recall the notion of a commitment scheme. A commitment scheme consists of three PPT algorithms:

- $\text{Setup}(1^\lambda) \rightarrow \text{ck}$: Given a security parameter λ , it generates a commitment key ck .
- $\text{Com}(\text{ck}, m) \rightarrow (c, \delta)$: Given a message m , it returns a commitment c and an opening δ .
- $\text{Open}(\text{ck}, c, m, \delta) \rightarrow b$: Given a commitment c , a message m , and an opening δ , it outputs 0 or 1.

E. DV-zkSNARKs

Designated Verifier Zero Knowledge Succinct Non-Interactive Arguments of Knowledge (DV-zkSNARKs) are a type of argument system that only a specific verifier, possessing a secret key, can verify the proof. A DV-zkSNARK is built from a relation $\mathbb{R}$ between a public statement $\mathbb{X}$ and a private witness $\mathbb{W}$. If the relation between $\mathbb{X}$ and $\mathbb{W}$ holds, then $(\mathbb{X}, \mathbb{W}) \in \mathbb{R}$. Formally, a DV-zkSNARK scheme Π for a relation $\mathbb{R}$ is composed of the triplet, defined as follows:

- $\text{Setup}(\mathbb{R}) \rightarrow (\mathbf{pk}, \mathbf{vk}, \mathbf{crs})$: It takes a relation $\mathbb{R}$, and outputs the prover key $\mathbf{pk}$, the verifier key $\mathbf{vk}$ and the $\mathbf{crs}$.
- $\text{Prover}(\mathbf{pk}, \mathbb{X}, \mathbb{W}, \mathbf{crs}) \rightarrow \pi$: It takes the prover key $\mathbf{pk}$, a public statement $\mathbb{X}$, a witness $\mathbb{W}$, and the $\mathbf{crs}$, then output the proof π implying that $(\mathbb{X}, \mathbb{W}) \in \mathbb{R}$.
- $\text{Verifier}(\mathbf{vk}, \mathbb{X}, \pi) \rightarrow b$: It takes the verification key $\mathbf{vk}$, the public statement $\mathbb{X}$, and the proof π , then outputs 0 or 1.

A DV-zkSNARK satisfies the properties given in the following three definitions.

Definition 2.2 (Completeness). *The probability*

$$\Pr\left[\begin{array}{l} (\mathbf{pk}, \mathbf{vk}, \mathbf{crs}) \leftarrow \text{Setup}(\mathbb{R}); \\ \pi \leftarrow \text{Prover}(\mathbf{pk}, \mathbf{crs}, \mathbb{X}, \mathbb{W}) \end{array} \mid \begin{array}{l} \text{Verifier}(\mathbf{vk}, \mathbb{X}, \pi) = 0 \\ \text{where } (\mathbb{X}, \mathbb{W}) \in \mathbb{R} \end{array}\right]$$

is negligible.

Definition 2.3 (Adaptive Knowledge Soundness). *For every efficient adversary $\mathcal{A}$, there exists an efficient extractor $\text{Ext}_\mathcal{A}$ with access to the internal state of $\mathcal{A}$ such that the probability*

$$\Pr\left[\begin{array}{l} ((\mathbf{pk}, \mathbf{vk}, \mathbf{crs}), \text{aux}) \leftarrow \text{Setup}(\mathbb{R}), \\ (\mathbb{X}, \pi) \leftarrow \mathcal{A}(\mathbb{R}, \text{aux}, (\mathbf{pk}, \mathbf{vk}, \mathbf{crs})), \\ \mathbb{W} \leftarrow \text{Ext}_\mathcal{A}(\mathcal{R}, \text{aux}, (\mathbf{pk}, \mathbf{vk})) \end{array} \mid \begin{array}{l} (\mathbb{X}, \mathbb{W}) \notin \mathbb{R}, \\ \text{Verifier}(\mathbf{vk}, \mathbb{X}, \pi) = 1 \end{array}\right]$$

is negligible, where $\mathbf{aux}$ is auxiliary input generated in setup phase.

The *adaptive* variant of this property means that the adversary $\mathcal{A}$ is allowed to observe the $\mathbf{crs}$ before choosing the statement and producing a proof. In contrast, *non-adaptive* knowledge soundness restricts the adversary to commit to a statement before observing the $\mathbf{crs}$.

It was shown in [16] (Lemma 6.1) that zkSNARKs constructed using linear targeted malleability assumption cannot achieve adaptive knowledge soundness.

Definition 2.4 (Zero-Knowledge). For every adversary $\mathcal{A}$ and $(\mathbb{X}, \mathbb{W}) \in \mathbb{R}$, there exists a simulator $\text{Sim}((\mathbf{pk}, \mathbf{vk}), \mathbf{aux}, \mathbb{X})$, such that the following equality holds:

$$\Pr \left[\begin{array}{l} ((\mathbf{pk}, \mathbf{vk}), \mathbf{aux}) \leftarrow \text{Setup}(\mathbb{R}), \\ \pi \leftarrow \text{Prover}(\mathbf{pk}, \mathbf{crs}, \mathbb{X}, \mathbb{W}) \end{array} \middle| \mathcal{A}((\mathbf{pk}, \mathbf{vk}), \mathbf{aux}, \mathbb{X}, \pi) = 1 \right] \\ \approx \\ \Pr \left[\begin{array}{l} ((\mathbf{pk}, \mathbf{vk}), \mathbf{aux}) \leftarrow \text{Setup}(\mathbb{R}), \\ \pi \leftarrow \text{Sim}((\mathbf{pk}, \mathbf{vk}), \mathbf{aux}, \mathbb{X}) \end{array} \middle| \mathcal{A}((\mathbf{pk}, \mathbf{vk}), \mathbf{aux}, \mathbb{X}, \pi) = 1 \right],$$

where $\mathbf{aux}$ is an auxiliary input produced by Setup.

F. Coordinate-Wise Special Soundness (CWSS)

To prove knowledge soundness, we rely on *coordinate-wise special soundness* (CWSS) as introduced in [29]. Let $\mathcal{C}$ be a finite set and $\ell \in \mathbb{N}$. For vectors $\vec{x} = (x_1, \dots, x_\ell)$ and $\vec{y} = (y_1, \dots, y_\ell)$ in $\mathcal{C}^\ell$, define the relation $\vec{x} \equiv_i \vec{y}$ for $i \in [\ell]$ as:

$$\vec{x} \equiv_i \vec{y} \iff x_i \neq y_i \text{ and } \forall j \in [\ell] \setminus \{i\}, x_j = y_j.$$

This means $\vec{x}$ and $\vec{y}$ differ only at the i -th position. Now define $\text{SS}(\mathcal{C}, \ell)$ as follows:

$$\left\{ (\vec{x}_1, \dots, \vec{x}_{\ell+1}) \in \mathcal{C}^{\ell+1} : \begin{array}{l} \exists k \in [\ell+1], \forall i \in [\ell], \\ \exists j \in [\ell+1] \setminus \{k\}, \vec{x}_k \equiv_i \vec{x}_j \end{array} \right\}.$$

Definition 2.5 (CWSS for three-round protocols).

Let $\Pi = (\text{Setup}, \text{Prover}, \text{Verifier})$ be a public-coin three-round interactive proof system for a relation $\mathbb{R}$, and suppose challenge space of $\mathcal{V}$ is $\mathcal{C}^\ell$. We say that Π satisfies ℓ -coordinate-wise special soundness if there exists a polynomial-time extractor that, given public parameters, a statement $\mathbb{X}$, and $\ell+1$ accepting transcripts $(\vec{c}_i, \pi_i)_{i \in [\ell+1]}$ with distinct $\vec{c}_i$ such that $(\vec{c}_1, \dots, \vec{c}_{\ell+1}) \in \text{SS}(\mathcal{C}, \ell)$, outputs a witness $\mathbb{W}$ such that $(\mathbb{X}, \mathbb{W}) \in \mathbb{R}$.

Lemma 2.6. [29] Let Π be a public-coin three-round protocol for relation $\mathbb{R}$ with the challenge space of $\mathcal{C}$. If Π is ℓ -coordinate-wise special sound, then it is knowledge sound with knowledge error $\frac{\ell}{|\mathcal{C}|}$.

G. Ring and Field Isomorphism

We make use of the isomorphism introduced in [14]. This isomorphism enables the conversion of vectors defined over a field into vectors over a ring. The following lemma formalizes this isomorphism.

Lemma 2.7. [14] Let b and r be positive integers such that $r|d$ and $p = b^r + 1$ is prime. Then $\mathcal{R}/(X^{d/r} - b)$ is isomorphic to $\mathbb{Z}_p^{d/r}$.

A notable feature of this isomorphism is that it reduces the norm of the encoded witness, bounding it by $\frac{b+2}{2}$, which is particularly advantageous for lattice-based problems. We denote the encoding and decoding algorithms by Ecd and Dcd , respectively.

III. REVISITING THE AJTAI COMMITMENT SCHEME

In this section, we introduce a set of modifications to the Ajtai commitment scheme. The primary modification lies in the setup phase, where we generate the commitment matrices using several uniformly random submatrices, each equipped with a trapdoor. The proposed scheme is shown in Figure 2. We refer to this modified scheme as the Trapdoor-based Ajtai Commitment Scheme (TACS). For hiding Ajtai commitment, we require two matrices $\mathbf{A}_{\text{com}}, \mathbf{A}_{\text{hide}} \in \mathcal{R}_q^{n(\log q + 1) \times nk}$. To ensure compactness, we assume $k > \log q + 1$.

Note that according to [30], each matrix $\mathbf{A}_i$ is statistically close to uniform. Since these matrices are generated independently using different trapdoors, the concatenated matrices $\mathbf{A}_{\text{com}}$ and $\mathbf{A}_{\text{hide}}$ follow a distribution that is close to uniform, as well. Therefore, the required conditions for both Ajtai matrices are still satisfied.

Trapdoor-based Ajtai Commitment Scheme (TACS)

Setup(1^λ)

- 1) Run $\text{TrapGen}(n, n \log q + n)$ independently $2k$ times to obtain matrices $\{\mathbf{A}_i, \mathbf{R}_i\}_{i=1}^{2k}$.
- 2) For each $\mathbf{A}_i$, split it into an upper and lower half:
$$\mathbf{A}_i^{(1)}, \mathbf{A}_i^{(2)} \in \mathcal{R}_q^{\frac{n}{2} \times (n \log q + n)}.$$

- 3) Construct the commitment matrices:

$$\mathbf{A}_{\text{com}} = \begin{bmatrix} (\mathbf{A}_1^{(1)})^\top & (\mathbf{A}_2^{(1)})^\top & \dots & (\mathbf{A}_{2k}^{(1)})^\top \end{bmatrix},$$

$$\mathbf{A}_{\text{hide}} = \begin{bmatrix} (\mathbf{A}_1^{(2)})^\top & (\mathbf{A}_2^{(2)})^\top & \dots & (\mathbf{A}_{2k}^{(2)})^\top \end{bmatrix}.$$

- 4) Return the commitment key $\mathbf{ck} = (\mathbf{A}_{\text{com}}, \mathbf{A}_{\text{hide}})$ and the trapdoor $\mathbf{td} = \{\mathbf{R}_i\}_{i=1}^{2k}$.

Commit($\mathbf{ck}, \vec{\mathbf{b}} \in \mathcal{R}_q^{nk}$)

- 1) Sample $\vec{\mu} \leftarrow \mathcal{D}_{s_\mu}^{nk}$.
- 2) Compute and return the commitment $\vec{\mathbf{c}} = \mathbf{A}_{\text{com}} \vec{\mathbf{b}} + \mathbf{A}_{\text{hide}} \vec{\mu}$.

Open($\mathbf{ck}, \vec{\mathbf{c}}, \vec{\mathbf{b}}, \vec{\mu}$)

- 1) Return 1 if and only if the following conditions hold:

$$\|\vec{\mathbf{b}}\|_\infty < \beta_{\text{open}}, \|\vec{\mu}\|_\infty < \beta_{\text{open}}, \vec{\mathbf{c}} = \mathbf{A}_{\text{com}} \vec{\mathbf{b}} + \mathbf{A}_{\text{hide}} \vec{\mu}.$$

Fig. 2. A Trapdoor-based Ajtai commitment scheme.

IV. AN ARGUMENT SYSTEM FOR VERIFYING INNER PRODUCT RELATION

In this section, we present an argument system for the relation $\mathbb{R}_{\text{IPA}}$ defined in Equation 1, built upon TACS commitment with a trusted setup. To construct this protocol, we first introduce several modifications to Figure 11. We then present the argument system for $\mathbb{R}_{\text{IPA}}$. This relation require proving two statement. First, the prover must demonstrate that the inner product has been computed correctly. Second, the prover must prove that the witness has a bounded norm.

A. Modified Preimage Sampling Algorithm

We modify the SamplePre algorithm to suit our protocol. In this algorithm, a trapdoor holder can sample a short preimage $\vec{u} = \vec{p} + [\frac{\mathbf{R}}{\mathbf{I}}] \vec{z}$ for a given syndrome vector $\vec{v}$. In our setting, however, the prover does not directly observe $\vec{u}$. As we see later, the vector $\vec{p}$ becomes part of the versifier's secret key, while a randomized version of $\vec{z}$ is embedded in the CRS. We note that this randomized vector is computationally independent of the trapdoor $\mathbf{R}$ and reveals no information about $\vec{z}$ itself. we introduce two key modifications to the standard preimage sampling algorithm.

- (1) Shared Perturbation Vector: For matrices $(\mathbf{A}_i, \mathbf{R}_i)$, $i \in [k]$, and corresponding syndromes $\vec{v}_i$, preimages $\vec{u}_i$ satisfying $\mathbf{A}_i \vec{u}_i = \vec{v}_i$ share the same perturbation vector $\vec{p}$.
- (2) Independent Sampling of $\vec{p}$ and $\vec{z}$: Vectors $\vec{p}$ and $\vec{z}$ are independently drawn from Gaussian distributions with distinct covariance matrices, ensuring $\vec{u}$ remains within the required norm bound. The modified SamplePre algorithm is shown in Figure 3.

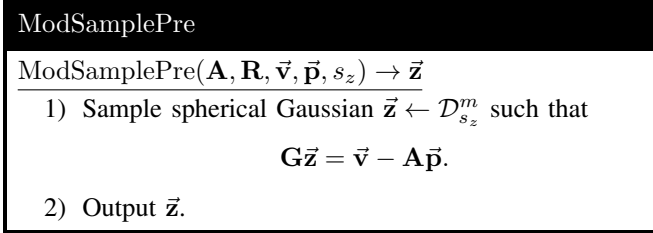


Fig. 3. A modified version of SamplePre Algorithm.

B. Inner product verification

In this section, we show that a verifier with trapdoors $\mathbf{R}_i$ for the TACS Ajtai matrices can verify the correctness of a claimed inner product between a committed witness and a uniformly random challenge $\vec{u}_{\text{chall}} \leftarrow \mathcal{U}(\mathcal{R}_q^{1 \times 2nk})$. We first generate a trapdoor vector corresponding to this challenge using the ChallTrap in Figure 4. We show that the verifier, equipped with the trapdoor vector and the secret $\vec{p}$, can efficiently verify the correctness of the inner product between the committed witness and the challenge vector $\vec{u}_{\text{chall}}$. It is easy to check that the vector $\vec{z}_G$ has a covariance matrix $ds_z^2 \sum_{i=0}^{\log q-1} 2^i \mathbf{I}$ where, s_z is the same as the input of Figure 3.

The following claim shows that a verifier who possesses $\vec{p}$ and having access to the product of $\vec{z}_G$ with the witness can verify the product of $\vec{u}_{\text{chall}}$ with the same witness.

Claim 4.1. *The following equality holds, if $\vec{z}_G$ is computed from $\vec{u}_{\text{chall}}$ using the algorithm ChallTrap.*

$$\vec{u}_{\text{chall}} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} = \langle \vec{p}, \vec{c} \rangle + \vec{z}_G \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix}. \quad (4)$$

Proof. Defining $\vec{u}_i$, $i \in [4k]$ as stated in step 1 of algorithm ChallTrap, we obtain:

$$\vec{u}_{\text{chall}} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} = \sum_{i=1}^{2k} (\vec{u}_i \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix})_{(i-1)\frac{n}{2}+1}^{h_i} + \vec{u}_{i+2k} \cdot [\vec{\mu}]_{(i-1)\frac{n}{2}+1}^{\frac{n}{2}}.$$

Lets set $l_i = (i-1)\frac{n}{2} + 1$ and $h_i = i\frac{n}{2}$ then we have:

$$\begin{aligned} \vec{u}_{\text{chall}} \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} &= \sum_{i=1}^{2k} \begin{bmatrix} \vec{u}_i^T \\ \vec{u}_{i+2k}^T \end{bmatrix}^T \begin{bmatrix} [\vec{b}]_{l_i}^{h_i} \\ [\vec{\mu}]_{l_i}^{h_i} \end{bmatrix} \\ &= \sum_{i=1}^{2k} (\mathbf{A}_i \vec{y}_i)^T \begin{bmatrix} [\vec{b}]_{l_i}^{h_i} \\ [\vec{\mu}]_{l_i}^{h_i} \end{bmatrix} \\ &= \sum_{i=1}^{2k} \vec{y}_i^T \left[(\mathbf{A}_i^{(1)})^T (\mathbf{A}_i^{(2)})^T \right] \begin{bmatrix} [\vec{b}]_{l_i}^{h_i} \\ [\vec{\mu}]_{l_i}^{h_i} \end{bmatrix}. \end{aligned}$$

We replace $\vec{y}_i$ vectors from step 2 of ChallTrap and get:

$$\begin{aligned} &\sum_{i=1}^{2k} (\vec{p}^T + \vec{z}_i^T [\mathbf{R}_i^T \quad \mathbf{I}]) \left[(\mathbf{A}_i^{(1)})^T (\mathbf{A}_i^{(2)})^T \right] \begin{bmatrix} [\vec{b}]_{l_i}^{h_i} \\ [\vec{\mu}]_{l_i}^{h_i} \end{bmatrix} \\ &= \vec{p}^T \sum_{i=1}^{2k} \left[(\mathbf{A}_i^{(1)})^T (\mathbf{A}_i^{(2)})^T \right] \begin{bmatrix} [\vec{b}]_{l_i}^{h_i} \\ [\vec{\mu}]_{l_i}^{h_i} \end{bmatrix} \\ &\quad + \sum_{i=1}^{2k} (\vec{z}_i^T [\mathbf{R}_i^T \quad \mathbf{I}]) \left[(\mathbf{A}_i^{(1)})^T (\mathbf{A}_i^{(2)})^T \right] \begin{bmatrix} [\vec{b}]_{l_i}^{h_i} \\ [\vec{\mu}]_{l_i}^{h_i} \end{bmatrix} \\ &= \vec{p}^T \cdot \vec{c} \\ &\quad + \sum_{i=1}^{2k} \vec{z}_i^T [\mathbf{R}_i^T \quad \mathbf{I}] \left((\mathbf{A}_i^{(1)})^T [\vec{b}]_{l_i}^{h_i} + (\mathbf{A}_i^{(2)})^T [\vec{\mu}]_{l_i}^{h_i} \right) \end{aligned}$$

Note that based on definition of gadget trapdoors we can write:

$$[\mathbf{R}_i^T \quad \mathbf{I}] (\mathbf{A}_i^{(1)})^T = (\mathbf{G}^{(1)})^T, \quad [\mathbf{R}_i^T \quad \mathbf{I}] (\mathbf{A}_i^{(2)})^T = (\mathbf{G}^{(2)})^T$$

Thus we obtain:

$$\begin{aligned} &= \vec{p}^T \cdot \vec{c} + \sum_{i=1}^{2k} \left(\vec{z}_i^T (\mathbf{G}^{(1)})^T [\vec{b}]_{l_i}^{h_i} + \vec{z}_i^T (\mathbf{G}^{(2)})^T [\vec{\mu}]_{l_i}^{h_i} \right) \\ &= \vec{p}^T \cdot \vec{c} + \sum_{i=1}^{2k} \left((\vec{z}_{G,i}^{(1)})^T [\vec{b}]_{l_i}^{h_i} + (\vec{z}_{G,i}^{(2)})^T [\vec{\mu}]_{l_i}^{h_i} \right) \\ &= \langle \vec{p}, \vec{c} \rangle + \vec{z}_G \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} \end{aligned}$$

□

The verifier must know $\vec{z}_G \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix}$ to check Equation 4, requiring an additional inner-product verification. A naive approach, sending $\vec{z}_G$ directly to the prover, however, creates a circular dependency, as each verification would require a fresh challenge $\vec{z}'_G$, and so on. To break this loop, the verifier embeds $\vec{z}_G$ into an apparently uniform vector using $\vec{u}_{\text{chall}}$. Specifically, they sample a secret value $\mathbf{z}_{\text{sec}}^{(1)}$ from a

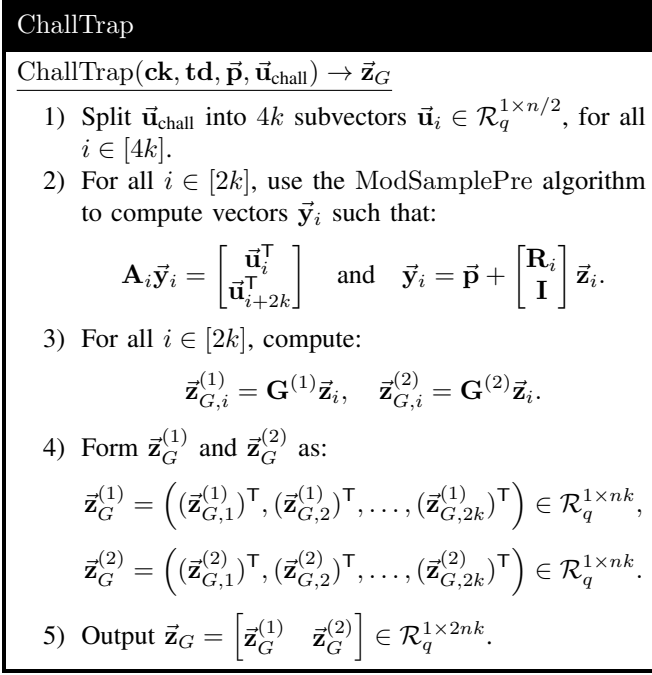


Fig. 4. Algorithm ChallTrap: Computing a trapdoor-dependent vector for a random challenge.

Gaussian distribution with covariance matrix $s_z^2 (\sum_{i=0}^{\log q-1} 2^i) \mathbf{I}$, which becomes part of their secret key. Then, the apply Gaussian2Uniform algorithm shown in Figure 5, to generate a close to uniform challenge vector $\vec{\mathbf{u}}_{z,\text{chall}}$ that masks $\vec{\mathbf{z}}_G$. This

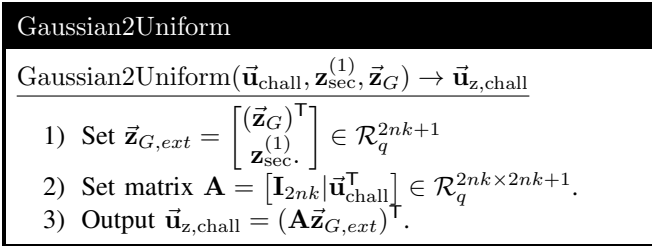


Fig. 5. The algorithm for masking a Gaussian vector.

algorithm uses the fact of Lemma 2.4. As a result, the derived vector $\vec{\mathbf{u}}_{z,\text{chall}}$, is computationally indistinguishable from a uniformly random vector over $\mathcal{R}_q^{1 \times 2nk}$. Note that $\vec{\mathbf{u}}_{z,\text{chall}}$ is equal to $\vec{\mathbf{z}}_G + \mathbf{z}_{\text{sec}}^{(1)} \vec{\mathbf{u}}_{\text{chall}}$. Therefore, the prover's response to this challenge is computed as

$$\vec{\mathbf{u}}_{z,\text{chall}} \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} = \vec{\mathbf{z}}_G \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} + \mathbf{z}_{\text{sec}}^{(1)} \left(\vec{\mathbf{u}}_{\text{chall}} \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} \right). \quad (5)$$

By adding $\langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle$ to both sides of Equation 5, and applying Claim 4.1, a verifier who possesses both $\mathbf{z}_{\text{sec}}^{(1)}$ and $\vec{\mathbf{p}}$ can efficiently check whether the following equality holds:

$$\vec{\mathbf{u}}_{z,\text{chall}} \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} + \langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle = (\mathbf{z}_{\text{sec}}^{(1)} + 1) \vec{\mathbf{u}}_{\text{chall}} \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}$$

It is important to note that it is computationally infeasible for a malicious prover to guess $\mathbf{z}_{\text{sec}}^{(1)}$, as it is sampled from a

Gaussian distribution with large variance. As a result, $\mathbf{z}_{\text{sec}}^{(1)}$ is statistically close to uniform, and the probability of correctly guessing it is bounded by $(\frac{1}{q})^d$.

C. Range Proof Verification

Beyond verifying the inner product, the verifier must be assured that the witness satisfies a norm constraint; otherwise, the commitment scheme's binding property may fail. For this, we use the approximate range proof technique from [27], [31], formalized in Lemma 2.3. As this method relies on inner products, it integrates naturally with our protocol. Since the original technique is defined over $\mathbb{Z}_q$, we adapt it to operate over a polynomial ring. In what follows, we describe how this adaptation is achieved. The witness vectors $\vec{\mathbf{b}}$ and $\vec{\mu}$ could be formed as matrices

$$\mathbf{B} = \begin{bmatrix} \text{Rot}(\vec{\mathbf{b}}[1]) & \text{Rot}(\vec{\mathbf{b}}[2]) & \dots & \text{Rot}(\vec{\mathbf{b}}[nk]) \end{bmatrix} \in \mathbb{Z}_q^{d \times dnk} \quad (6)$$

$$\mathbf{MU} = \begin{bmatrix} \text{Rot}(\vec{\mu}[1]) & \text{Rot}(\vec{\mu}[2]) & \dots & \text{Rot}(\vec{\mu}[nk]) \end{bmatrix} \in \mathbb{Z}_q^{d \times dnk} \quad (7)$$

$$\mathbf{M} = [\mathbf{B} | \mathbf{MU}] \in \mathbb{Z}_q^{d \times 2dnk} \quad \|\mathbf{M}\|_\infty \leq \beta_{\text{open}}$$

For a uniformly sampled matrix $\mathbf{K}$ from $\{0,1\}^{2dnk \times \lambda}$, the probability that a malicious prover can find a matrix $\mathbf{M}$ such that $\|\mathbf{MK}\|_\infty < \frac{1}{2}\beta_{\text{open}}$ is bounded by $2^{-\lambda}$. It is necessary to sample these vectors using a hash function, as they will later be employed for extraction, which is only feasible within the random oracle model [29]. Each column of $\mathbf{K}$ can be interpreted as a vector $\vec{\mathbf{k}}_i \in \mathcal{R}_q^{2nk}$, obtained by applying the inverse coefficient embedding function, $\text{cf}(\cdot)^{-1}$, on every d consecutive entries in that column. The matrix product $\mathbf{MK}$ is equivalent to $\begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}^\top \cdot \begin{bmatrix} \vec{\mathbf{k}}_1 & \vec{\mathbf{k}}_2 & \dots & \vec{\mathbf{k}}_\lambda \end{bmatrix}$. Therefore, these vectors $\vec{\mathbf{k}}_i$ can naturally serve as norm-checking challenge vectors. One can apply ChallTraP to these vector to generate the corresponding trapdoor challenges, denoted $\vec{\mathbf{z}}_{G,\text{chall}}^{(i)}$. The prover's response to such a challenge is sufficient to compute the inner product $\vec{\mathbf{k}}_i^\top \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}$, as established in Claim 4.1. In other words, we have:

$$\vec{\mathbf{k}}_i^\top \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix} = \langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle + \vec{\mathbf{z}}_{G,\text{chall}}^{(i)} \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}. \quad (8)$$

Thus, the verifier can compute the left-hand side of Equation 8 by adding $\langle \vec{\mathbf{p}}, \vec{\mathbf{c}} \rangle$ to the prover's response to the trapdoor challenge. However, note that the vectors $\vec{\mathbf{z}}_{G,\text{chall}}^{(i)}$ must be hidden from the prover. Given that these vectors follow a Gaussian distribution, the Gaussian2Uniform algorithm can be applied to convert them into vectors that are statistically close to uniform. To do this, we use each $\vec{\mathbf{z}}_{G,\text{chall}}^{(i)}$, together with a newly sampled secret $\mathbf{z}_{\text{sec}}^{(i)}$ and the verifier's original challenge vector $\vec{\mathbf{u}}_{\text{chall}}$, as inputs to the Gaussian2Uniform algorithm. This algorithm yields a uniformly distributed vector $\vec{\mathbf{u}}_{\text{chall}}^{(i)}$ which is equal to $\vec{\mathbf{z}}_{G,\text{chall}}^{(i)} + \mathbf{z}_{\text{sec}}^{(i)} \vec{\mathbf{u}}_{\text{chall}}$. This can be sent to the prover as plain. The prover's response to this challenge is $\vec{\mathbf{u}}_{\text{chall}}^{(i)} \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}$. From the verifier's view, this response is equal to $(\vec{\mathbf{z}}_{G,\text{chall}}^{(i)} + \mathbf{z}_{\text{sec}}^{(i)} \vec{\mathbf{u}}_{\text{chall}}) \cdot \begin{bmatrix} \vec{\mathbf{b}} \\ \vec{\mu} \end{bmatrix}$. The verifier can then

The DV-LatticeIPA Protocol

Setup_{CRS}(1^λ)

- 1) Run TACS. Setup algorithm and derive $\mathbf{ck}$ and $\mathbf{td}$.
- 2) Parse $\mathbf{ck} := (\mathbf{A}_{\text{com}}, \mathbf{A}_{\text{hide}})$ and $\mathbf{td} := \{\mathbf{R}_i\}_{i=1}^{2k}$.
- 3) Sample $\mathbf{z}_{\text{sec}}^{(0)} \leftarrow \mathcal{D}_{s_z \sum_{i=0}^{\log q-1} 2^i}$, $\tilde{\mathbf{u}}_{\text{chall}}^{(0)} \leftarrow \mathcal{U}(\mathcal{R}_q^{1 \times 2nk})$ and $\tilde{\mathbf{p}} \leftarrow \mathcal{D}_{s_p}^{n(\log q+1)}$.
- 4) $\tilde{\mathbf{z}}_{\text{G,verify}}^{(0)} \leftarrow \text{ChallTrap}(\mathbf{ck}, \mathbf{td}, \tilde{\mathbf{p}}, \tilde{\mathbf{u}}_{\text{chall}}^{(0)})$, and $\tilde{\mathbf{u}}_{\text{G,verify}}^{(0)} \leftarrow \text{Gaussian2Uniform}(\tilde{\mathbf{u}}_{\text{chall}}^{(0)}, \mathbf{z}_{\text{sec}}^{(0)}, \tilde{\mathbf{z}}_{\text{G,verify}}^{(0)})$.
- 5) Sample $\mathbf{K} \leftarrow \{0, 1\}^{2dnk \times \lambda}$ using a hash function $H(\cdot)$.
- 6) For $j = 1, 2, \dots, \lambda$ do:
- 7) sample $\mathbf{z}_{\text{sec}}^{(j)}, \mathbf{z}_{\text{sec}}^{(j+\lambda)} \leftarrow \mathcal{D}_{s_z \sum_{i=0}^{\log q-1} 2^i}$.
- 8) convert the j th column of $\mathbf{K}$ to $\mathbf{k}_j \in \mathcal{R}_q^{2nk}$ by applying $\text{cf}^{-1}(\cdot)$ to each d consecutive entries in this column.
- 9) $\tilde{\mathbf{z}}_{\text{G,chall}}^{(j)} \leftarrow \text{ChallTrap}(\mathbf{ck}, \mathbf{td}, \tilde{\mathbf{p}}, \mathbf{k}_j^T)$, and $\tilde{\mathbf{u}}_{\text{chall}}^{(j)} \leftarrow \text{Gaussian2Uniform}(\tilde{\mathbf{u}}_{\text{chall}}^{(0)}, \mathbf{z}_{\text{sec}}^{(j+\lambda)}, \tilde{\mathbf{z}}_{\text{G,chall}}^{(j)})$.
- 10) $\tilde{\mathbf{z}}_{\text{G,verify}}^{(j)} \leftarrow \text{ChallTrap}(\mathbf{ck}, \mathbf{td}, \tilde{\mathbf{p}}, \tilde{\mathbf{u}}_{\text{chall}}^{(j)})$, $\tilde{\mathbf{u}}_{\text{G,verify}}^{(j)} \leftarrow \text{Gaussian2Uniform}(\tilde{\mathbf{u}}_{\text{chall}}^{(1)}, \mathbf{z}_{\text{sec}}^{(j)}, \tilde{\mathbf{z}}_{\text{G,verify}}^{(j)})$.
- 11) Set matrices $\mathbf{U}_{\text{chall}}, \mathbf{U}_{\text{verify}}, \tilde{\mathbf{z}}_{\text{sec}}$ as follows:

$$\mathbf{U}_{\text{chall}} = \begin{bmatrix} \tilde{\mathbf{u}}_{\text{chall}}^{(0)} \\ \tilde{\mathbf{u}}_{\text{chall}}^{(1)} \\ \vdots \\ \tilde{\mathbf{u}}_{\text{chall}}^{(\lambda)} \end{bmatrix}, \quad \mathbf{U}_{\text{verify}} = \begin{bmatrix} \tilde{\mathbf{u}}_{\text{G,verify}}^{(0)} \\ \tilde{\mathbf{u}}_{\text{G,verify}}^{(1)} \\ \vdots \\ \tilde{\mathbf{u}}_{\text{G,verify}}^{(\lambda)} \end{bmatrix}, \quad \tilde{\mathbf{z}}_{\text{sec}} = \begin{bmatrix} \mathbf{z}_{\text{sec}}^{(0)} \\ \mathbf{z}_{\text{sec}}^{(1)} \\ \vdots \\ \mathbf{z}_{\text{sec}}^{(2\lambda)} \end{bmatrix}$$

- 12) Output $\mathbf{ck} = (\mathbf{A}_{\text{com}}, \mathbf{A}_{\text{hide}})$, $\mathbf{vk} = (\tilde{\mathbf{p}}, \tilde{\mathbf{z}}_{\text{sec}})$, $\mathbf{crs} = (\mathbf{U}_{\text{chall}}, \mathbf{U}_{\text{verify}})$.

$\mathcal{P}(\mathbf{ck}, \mathbf{crs}, \tilde{\mathbf{b}}, \tilde{\boldsymbol{\mu}})$

- 1) Run TACS. Commit($\mathbf{ck}, \tilde{\mathbf{b}}, \tilde{\boldsymbol{\mu}}$) and derive $\tilde{\mathbf{c}}$.
- 2) Parse $\mathbf{crs} := \{\mathbf{U}_{\text{chall}}, \mathbf{U}_{\text{verify}}\}$.
- 3) Compute $\tilde{\boldsymbol{\pi}}_{\text{chall}} = \mathbf{U}_{\text{chall}} \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\boldsymbol{\mu}} \end{bmatrix} \in \mathcal{R}_q^{\lambda+1}$ and $\tilde{\boldsymbol{\pi}}_{\text{verify}} = \mathbf{U}_{\text{verify}} \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\boldsymbol{\mu}} \end{bmatrix} \in \mathcal{R}_q^{\lambda+1}$.
- 4) Send $(\tilde{\mathbf{c}}, \tilde{\boldsymbol{\pi}}_{\text{chall}}, \tilde{\boldsymbol{\pi}}_{\text{verify}})$ to the verifier.

$\mathcal{V}(\mathbf{crs}, \mathbf{vk})$

- 1) Return 1 if and only if:

$$\tilde{\boldsymbol{\pi}}_{\text{verify}} + \begin{bmatrix} \langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle \\ \langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle \\ \vdots \\ \langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle \end{bmatrix} = \tilde{\boldsymbol{\pi}}_{\text{chall}}[1] \begin{bmatrix} \tilde{\mathbf{z}}_{\text{sec}}[1]+1 \\ \tilde{\mathbf{z}}_{\text{sec}}[2]+1 \\ \vdots \\ \tilde{\mathbf{z}}_{\text{sec}}[\lambda+1]+1 \end{bmatrix} \quad (9)$$

$$\| \text{cf}(\tilde{\boldsymbol{\pi}}_{\text{chall}}[j] + \langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle - \tilde{\mathbf{z}}_{\text{sec}}[\lambda+j]\tilde{\boldsymbol{\pi}}_{\text{chall}}[1]) \|_\infty \leq \frac{1}{2}\beta_{\text{open}} \text{ for } j = 2, 3, \dots, \lambda+1 \quad (10)$$

Fig. 6. Designated Verifier Lattice-based Inner product Argument System.

use the previously verified inner product between $\tilde{\mathbf{u}}_{\text{chall}}$ and the witness, together with the secret $\mathbf{z}_{\text{sec}}^{(i)}$, to extract the inner product between $\tilde{\mathbf{z}}_{\text{G,chall}}^{(i)}$ and the witness. Furthermore, the verifier must be convinced that the value of $\tilde{\mathbf{u}}_{\text{chall}}^{(i)} \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\boldsymbol{\mu}} \end{bmatrix}$ is computed correctly by the prover. To ensure this, the verifier applies the same technique described in Section IV-B to verify the correctness of the inner product.

Putting everything together, we obtain an argument system that can be used to verify the inner product of a committed low-norm witness with a uniformly sampled random challenge, as defined in Equation 1. This argument system is illustrated in Figure 6.

V. SECURITY ANALYSIS

This section analyzes the security of the proposed argument system, focusing on completeness, knowledge soundness, and

zero-knowledge.

Theorem 5.1 (Completeness). *The DV-LatticeIPA protocol satisfies the completeness property with probability $1 - 2^{-\lambda}$.*

Proof. To prove completeness, we begin with Equation 9. For simplicity, we show that this equality holds for the first two rows of the challenge matrices, as the remaining $\lambda - 1$ rows follow similarly from the same structure. We begin with the first row. By definition, the first entry of $\tilde{\boldsymbol{\pi}}_{\text{chall}}$ is $\tilde{\mathbf{u}}_{\text{chall}}^{(0)} \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\boldsymbol{\mu}} \end{bmatrix}$. According to Claim 4.1, this is equal to $\langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle + \tilde{\mathbf{z}}_{\text{G,verify}}^{(0)} \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\boldsymbol{\mu}} \end{bmatrix}$. On the other hand $\tilde{\boldsymbol{\pi}}_{\text{verify}}[1]$ is:

$$\begin{aligned} \tilde{\mathbf{u}}_{\text{G,verify}}^{(0)} \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\boldsymbol{\mu}} \end{bmatrix} &= \left(\begin{bmatrix} \mathbf{I}_{2nk} | \tilde{\mathbf{u}}_{\text{chall}}^{(0)} \end{bmatrix} \begin{bmatrix} (\tilde{\mathbf{z}}_{\text{G,verify}}^{(0)})^\top \\ \mathbf{z}_{\text{sec}}^{(0)} \end{bmatrix} \right)^\top \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\boldsymbol{\mu}} \end{bmatrix} \\ &= \left((\tilde{\mathbf{z}}_{\text{G,verify}}^{(0)})^\top + \mathbf{z}_{\text{sec}}^{(0)} (\tilde{\mathbf{u}}_{\text{chall}}^{(0)})^\top \right)^\top \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\boldsymbol{\mu}} \end{bmatrix} \end{aligned}$$

$$\begin{aligned}
&= \left((\vec{z}_{G,\text{verify}}^{(0)})^\top + \mathbf{z}_{\text{sec}}^{(0)} (\vec{u}_{\text{chall}}^{(0)})^\top \right)^\top \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} \\
&= \vec{z}_{G,\text{verify}}^{(0)} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} + \mathbf{z}_{\text{sec}}^{(0)} \vec{u}_{\text{chall}}^{(0)} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} \\
&= \vec{z}_{G,\text{verify}}^{(0)} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} + \mathbf{z}_{\text{sec}}^{(0)} \vec{\pi}_{\text{chall}}[1]
\end{aligned}$$

Thus, we conclude that:

$$\vec{\pi}_{\text{verify}}[1] + \langle \vec{p}, \vec{c} \rangle = (1 + \mathbf{z}_{\text{sec}}^{(0)}) \vec{\pi}_{\text{chall}}[1]$$

The same reasoning applies to the second row and extends naturally to all rows in Equation 9.

Now we prove that Equation 10 holds with high probability for an honest prover:

$$\vec{\pi}_{\text{chall}}[j] = \vec{u}_{\text{chall}}^{(j-1)} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} = \vec{z}_{G,\text{chall}}^{(j-1)} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} + \mathbf{z}_{\text{sec}}^{(j-1)} \vec{\pi}_{\text{chall}}[1] \rightarrow$$

$$\vec{\pi}_{\text{chall}}[j] - \mathbf{z}_{\text{sec}}^{(j-1)} \vec{\pi}_{\text{chall}}[1] = \vec{z}_{G,\text{chall}}^{(j-1)} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} \quad 2 \leq j \leq \lambda + 1. \quad (11)$$

Since $\vec{z}_{G,\text{chall}}^{(j-1)}$ is obtained by applying ChallTrap to the vector $\vec{k}_j^\top$, Claim 4.1, Equation 4 implies that $\vec{k}_j^\top \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} = \langle \vec{p}, \vec{c} \rangle + \vec{z}_{G,\text{chall}}^{(j-1)} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix}$. Therefore, by adding $\langle \vec{p}, \vec{c} \rangle$ to Equation 11 we have:

$$\vec{\pi}_{\text{chall}}[j] - \mathbf{z}_{\text{sec}}^{(j-1)} \vec{\pi}_{\text{chall}}[1] + \langle \vec{p}, \vec{c} \rangle = \vec{k}_j^\top \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix}$$

If the prover uses short vectors, then the l_∞ -norm of the resulting value is strictly less than $\frac{1}{2}\beta_{\text{open}}$. Overall, the probability that this condition fails for any of the λ challenge vectors $\vec{k}_j$ is at most $1 - 2^{-\lambda}$. $\square$

We next prove coordinate-wise special soundness (CWSS), which implies knowledge soundness. In the standard CWSS proof, the prover fixes a witness while the extractor issues multiple challenges and uses the accepted transcripts to recover it. In our setting, each execution yields a fresh commitment, allowing a malicious prover to change the witness between runs. To overcome this, the extractor includes a verification step to ensure, with high probability, that the same witness is used across executions. Otherwise, it aborts. The following claim, shows this possibility.

Claim 5.1. *Let the witness have length $2nk$. If the extractor invokes the DV-LatticeIPA argument $2nk + 1$ times, the probability that a malicious prover employs different witnesses in any execution while still producing accepting transcripts is at most $\frac{2nk+1}{q^d}$.*

Proof. Let the $\mathbf{ck}_i$, $\mathbf{vk}_i$, and $\mathbf{crs}_i$ for all $i \in [2nk + 1]$ denote the outputs of the setup phase for each execution of the protocol. The matrices $\mathbf{U}_{\text{verify},i}$ are sampled independently and have uniform distribution. For any fixed row, say, the r th row, consider the vectors $\vec{u}_{\text{verify},i}^{(r-1)}$ for all $i \in [2nk + 1]$, each of length $2nk$. Since we have $2nk + 1$ such vectors in a $2nk$ dimensional space, they must be linearly dependent. Now, select one of these challenge vectors at random, say the j th one. The remaining $2nk$ vectors form a linearly independent

set and thus span the space. Therefore, the j th challenge vector can be written as a linear combination of the others. Let $\mathbf{h}_i$ denote the coefficients of this linear combination. Then we have:

$$\begin{aligned}
\vec{u}_{\text{verify},j}^{(r-1)} &= \sum_{i=1, i \neq j}^{2nk+1} \mathbf{h}_i \vec{u}_{\text{verify},i}^{(r-1)} \times \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} \\
\vec{u}_{\text{verify},j}^{(r-1)} \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} &= \sum_{i=1, i \neq j}^{2nk+1} \mathbf{h}_i \vec{u}_{\text{verify},i}^{(r-1)} \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} \\
\vec{\pi}_{\text{verify},j}[r] &= \sum_{i=1, i \neq j}^{2nk+1} \mathbf{h}_i \vec{\pi}_{\text{verify},i}[r]. \quad (12)
\end{aligned}$$

This equality implies that if the prover uses the same witness across all executions, the response to the j th challenge must be a linear combination of the responses to the other challenges with the same coefficients used for challenge vectors. If this consistency check fails, the extractor aborts. We next show that if the prover deviates from the witness in any execution, Equation 12 holds with only negligible probability. W.L.O.G we assume that $r > 1$. Then we could replace $\vec{\pi}_{\text{verify},i}[r]$ in Equation 12 with the r th entry of Equation 9 and obtain:

$$\begin{aligned}
&(\vec{z}_{\text{sec},j}[r] + 1) \vec{\pi}_{\text{chall},j}[1] - \langle \vec{p}_j, \vec{c}_j \rangle = \\
&\sum_{i=1, i \neq j}^{2nk+1} \mathbf{h}_i ((\vec{z}_{\text{sec},i}[r] + 1) \vec{\pi}_{\text{chall},i}[1] - \langle \vec{p}_i, \vec{c}_i \rangle).
\end{aligned}$$

We define a multilinear function $g^{(j)}(\cdot)$ as follow:

$$\begin{aligned}
g^{(j)}(\vec{x}) &= \sum_{i=1, i \neq j}^{2nk+1} \mathbf{h}_i (\vec{\pi}_{\text{chall},i}[1] (\mathbf{x}_i + 1) - \langle \vec{p}_i, \vec{c}_i \rangle) \\
&\quad - (\mathbf{x}_j + 1) \vec{\pi}_{\text{chall},j}[1] + \langle \vec{p}_j, \vec{c}_j \rangle. \quad (13)
\end{aligned}$$

It is obvious that:

$$g^{(j)}([\vec{z}_{\text{sec},1}[r], \dots, \vec{z}_{\text{sec},2nk+1}[r]]) = 0.$$

Since the values $\vec{z}_{\text{sec},i}[r]$ are sampled from a Gaussian distribution with a parameter significantly larger than q , they can be treated as being uniformly distributed over $\mathcal{R}_q$. In the other words, the evaluation of $g^{(j)}(\cdot)$ at a uniformly sampled random point becomes zero. If a malicious prover uses a different witness in at least one execution of the protocol, then the values of $\vec{\pi}_{\text{chall},i}[1]$ or $\vec{c}_i$ changes for at least one $i \in [2nk + 1]$. This means that the function $g^{(j)}(\cdot)$ changes to another function say $g^{(j)'}(\cdot)$. According to generalized Schwartz-Zippel Lemma, 2.2), the probability that the evaluation of $g^{(j)'}(\vec{x})$ at random point becomes zero is at most $\frac{2nk+1}{q^d}$. $\square$

The intuition behind proving CWSS is to leverage the challenge vectors used in the range proof to extract the witness. Since these challenges are embedded within the $\mathbf{U}_{\text{chall}}$ matrices, we utilize the responses to these challenges to first derive the response corresponding to the actual range proof challenge, and subsequently extract the witness from it.

Theorem 5.2 (CWSS). *The DV-LatticeIPA protocol satisfies coordinate-wise special soundness property.*

The simulator algorithm $\mathcal{S}_{\text{IPA}}$

$\mathcal{S}_{\text{IPA}}(\text{ck}, \text{crs}, \text{vk})$

- 1) Sample $\vec{c} \leftarrow \mathcal{U}(\mathcal{R}_q^{n \log q + n})$.
- 2) Sample $\vec{\pi}_{\text{chall}}[1] \leftarrow \mathcal{U}(\mathcal{R}_q)$ and compute $\vec{\pi}_{\text{verify}}[1]$ as follows:

$$\vec{\pi}_{\text{verify}}[1] = (\vec{z}_{\text{sec}}[1] + 1)\vec{\pi}_{\text{chall}}[1] - \langle \vec{p}, \vec{c} \rangle$$
- 3) For $j = 2, 3, \dots, \lambda + 1$ do:
- 4) Fix π'_j where $\|cf(\pi'_j)\|_\infty \leq \frac{1}{2}\beta_{\text{open}}$ and computes $\vec{\pi}_{\text{chall}}[j]$ and $\vec{\pi}_{\text{verify}}[j]$ as follows:

$$\vec{\pi}_{\text{chall}}[j] = \pi'_j + \vec{z}_{\text{sec}}[j]\vec{\pi}_{\text{chall}}[1] - \langle \vec{p}, \vec{c} \rangle, \quad \vec{\pi}_{\text{verify}}[j] = (\vec{z}_{\text{sec}}[\lambda + j] + 1)\vec{\pi}_{\text{chall}}[1] - \langle \vec{p}, \vec{c} \rangle$$
- 5) Output $(\vec{c}, \vec{\pi}_{\text{chall}}, \vec{\pi}_{\text{verify}})$

Fig. 7. Simulator algorithm for the DV-LatticeIPA Argument system.

Proof. Let $\ell = 2nk$. To apply the CWSS extractor, we require $\ell + 1$ accepting transcripts of the DV-LatticeIPA protocol:

$$(\mathbf{U}_{\text{chall},i}, \mathbf{U}_{\text{verify},i}, \vec{c}_i) \quad \forall i \in [2nk + 1].$$

First, the verifier ensures that the polynomial $g^{(j)}(\vec{x})$ defined in Claim 5.1 evaluates to zero at a randomly chosen index $r > 1$, where $\vec{x} = [\vec{z}_{\text{sec},1}[r], \dots, \vec{z}_{\text{sec},2nk+1}[r]]$. Let $\vec{\pi}_{\text{chall},i}$ and $\mathbf{U}_{\text{chall},i}$ denote the vectors and matrices obtained by removing the first row of $\vec{\pi}_{\text{chall},i}$ and $\mathbf{U}_{\text{chall},i}$, respectively. We use $(\mathbf{U}_{\text{chall},i}, \vec{\pi}_{\text{chall},i})_{i \in [2nk+1]}$ for extraction. Each $\mathbf{U}_{\text{chall},i}$ corresponds to a random matrix $\mathbf{K}_{\mathcal{R},i} = [\vec{k}_{1,i} \quad \vec{k}_{2,i} \quad \dots \quad \vec{k}_{\lambda,i}]$. Replacing $\mathbf{K}_{\mathcal{R},i}$ in Equation 10, results in:

$$\vec{\pi}_{\text{chall},i} + \begin{bmatrix} \langle \vec{p}_i, \vec{c}_i \rangle \\ \langle \vec{p}_i, \vec{c}_i \rangle \\ \vdots \\ \langle \vec{p}_i, \vec{c}_i \rangle \end{bmatrix} - \begin{bmatrix} \vec{z}_{\text{sec},i}[2] \\ \vec{z}_{\text{sec},i}[3] \\ \vdots \\ \vec{z}_{\text{sec},i}[\lambda+1] \end{bmatrix} \vec{\pi}_{\text{chall},i}[1] = \mathbf{K}_{\mathcal{R},i}^\top \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix}. \quad (14)$$

By applying the coefficient-extraction function $cf(\cdot)$ to each entry of the columns of $\mathbf{K}_{\mathcal{R},i}$, we obtain the matrix $\mathbf{K}_i \in \{0, 1\}^{2dnk \times \lambda}$ as sampled in Step 5 of the protocol in Figure 6. Therefore, the right hand side of Equation 14 is equivalent to $([\mathbf{B} \mid \mathbf{MU}]\mathbf{K}_i)^\top$, where $\mathbf{B}$ and $\mathbf{MU}$ are defined in Equations 6 and 7, respectively.

Without loss of generality, assume that $\mathbf{K}_{\mathcal{R},1}$ and $\mathbf{K}_{\mathcal{R},i}$ differ in exactly i 'th entry of a chosen column. Each entry of $\mathbf{K}_{\mathcal{R},i}$ is of the form $\sum_{j=0}^{d-1} v_j X^j$ with $v_j \in \{0, 1\}$. Thus, $\mathbf{K}_i - \mathbf{K}_1$ is zero in all entries except in i 'th, which equals $\pm X^i$. Consequently

$$\mathbf{K}_{\mathcal{R},i}^\top \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} - \mathbf{K}_{\mathcal{R},1}^\top \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} = \pm X^i \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} [i].$$

As a result, the extractor can recover each coordinate of the witness, thereby completing the extraction. $\square$

Next, we prove the zero-knowledge property. To this end, we first establish two supporting claims.

Claim 5.2. *The inner product $\langle \vec{p}, \vec{c} \rangle$ is statistically close to uniform over $\mathcal{R}_q$, assuming that the commitment $\vec{c}$ is generated honestly and sampling parameter of the first component of verifier's secret key, s_p satisfies $s_p > 2d \cdot q^{\frac{1}{n \log q + n} + \frac{2}{d(n \log q + n)}}$.*

Proof. It was shown in [32] that honestly generated Ajtai commitment $\vec{c}$ is statistically indistinguishable from a uniformly random vector over $\mathcal{R}_q^m$, where $m = n \log q + n$. We can view $\vec{c}^\top$ as a $1 \times m$ row vector. If $\vec{p}$ is sampled from a discrete Gaussian distribution with standard deviation s_p satisfying the given bound, then the conditions of Lemma 2.5 are met. As a result, the inner product $\vec{c}^\top \cdot \vec{p}$ is statistically close to uniform over $\mathcal{R}_q$. $\square$

Claim 5.3. *Each row of the vectors $\vec{\pi}_{\text{chall}}$ and $\vec{\pi}_{\text{verify}}$ is statistically indistinguishable from a uniformly random vector, assuming the following conditions hold: the vector $\vec{\mu}$ is sampled from $\mathcal{D}_\sigma^{nk}$ where $\sigma > 2d \cdot q^{\frac{1}{nk} + \frac{2}{d \cdot nk}}$, q is a prime such that $q \equiv 5 \pmod{8}$, and $\lambda + 1 < nk$.*

Proof. The matrix $\mathbf{U}_{\text{chall}}$ is sampled uniformly at random. The response vector is computed as:

$$\vec{\pi}_{\text{chall}} = \mathbf{U}_{\text{chall}} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} = \mathbf{U}_{\text{chall}}^{(1)} \cdot \vec{b} + \mathbf{U}_{\text{chall}}^{(2)} \cdot \vec{\mu},$$

where $\mathbf{U}_{\text{chall}}^{(2)} \in \mathcal{R}_q^{(\lambda+1) \times nk}$ denotes the second half of $\mathbf{U}_{\text{chall}}$.

Since $\lambda + 1 < nk$, and $\vec{\mu}$ is sampled from a sufficiently wide Gaussian, Lemma 2.5 implies that $\mathbf{U}_{\text{chall}}^{(2)} \cdot \vec{\mu}$ is statistically close to uniform. As a result, it masks the deterministic term $\mathbf{U}_{\text{chall}}^{(1)} \cdot \vec{b}$, making the entire response $\vec{\pi}_{\text{chall}}$ statistically indistinguishable from uniform. The same reasoning applies to $\vec{\pi}_{\text{verify}}$, which has an analogous structure. $\square$

Theorem 5.3 (Zero-Knowledge). *The DV-LatticeIPA argument system satisfies the zero-knowledge property, assuming that: the simulator $\mathcal{S}_{\text{IPA}}$, as defined in Figure 7, is used. q is a prime such that $q \equiv 5 \pmod{8}$. The noise vector $\vec{\mu}$ is sampled from $\mathcal{D}_\sigma$ where $\sigma > 2d \cdot q^{\frac{1}{nk} + \frac{2}{d \cdot nk}}$.*

Proof. According to Claim 5.3 and the statistical hiding property of the commitment, the three components of the transcript, $(\vec{c}, \vec{\pi}_{\text{chall}}, \vec{\pi}_{\text{verify}})$, produced by the honest prover are statistically close to uniform. It suffices to show that the corresponding components generated by the simulator in Figure 7, are also statistically close to uniform. In that case,

no distinguisher, as defined in Definition 2.4, can distinguish between the output of the simulator and that of an honest prover.

Commitment $\vec{c}$: According to Step 1, the commitment $\vec{c}$ is sampled directly from a uniform distribution over $\mathcal{R}_q^m$.

Challenge responses $\vec{\pi}_{\text{chall}}$: In Step 2, the first entry $\vec{\pi}_{\text{chall}}[1]$ is sampled uniformly at random over $\mathcal{R}_q$. The remaining entries are computed as:

$$\vec{\pi}_{\text{chall}}[j] = \pi'_j + \vec{z}_{\text{sec}}[j] \cdot \vec{\pi}_{\text{chall}}[1] - \langle \vec{p}, \vec{c} \rangle \quad \forall j \in [2, \lambda + 1],$$

where π'_j is a fixed (deterministic) value and does not increase entropy. The inner product $\langle \vec{p}, \vec{c} \rangle$ is statistically close to uniform by Claim 5.2. It remains to show that $\vec{z}_{\text{sec}}[j] \cdot \vec{\pi}_{\text{chall}}[1]$ is statistically close to uniform. Since $\vec{\pi}_{\text{chall}}[1]$ is uniformly distributed and can be viewed as a 1×1 matrix, and $\vec{z}_{\text{sec}}[j]$ is sampled from a wide enough Gaussian distribution, Lemma 2.5 applies to it. Therefore, this term is statistically close to uniform, and so is each $\vec{\pi}_{\text{chall}}[j]$.

Verification responses $\vec{\pi}_{\text{verify}}$: An identical argument applies to the entries of $\vec{\pi}_{\text{verify}}$, which are constructed in the same form under the same conditions. Therefore, all components generated by the simulator are statistically close to uniform, and thus indistinguishable from those in a real protocol execution. $\square$

VI. A DV-ZKSNARK BASED ON DV-LATTICEIPA PROTOCOL

In this section, we introduce a new variant of lattice-based DV-zkSNARKs based on the DV-LatticeIPA protocol. The LPCP compiler [18], is used to construct DV-zkSNARKs. It involves four challenges, which the prover responds by computing their inner products with the witness. In prior works [18], [23], these challenges are encrypted via homomorphic encryption and embedded in the CRS. In contrast, our approach uses the DV-LatticeIPA protocol. By applying the technique from Figure 1, we embed the LPCP query matrix challenges into statistically uniform-looking vectors. The DV-LatticeIPA protocol then allows the verifier to succinctly verify the prover's responses to these challenges. The proposed construction is shown in Figure 8.

To ensure compatibility with ring arithmetics, we need to introduce several modifications in the setup phase of the LPCP protocol and incorporate these modifications into the setup phase of DV-LatticeIPA protocol. Additionally, adjustments are required in the prover and verifier algorithms. The prover first encodes each entry of the witness using the Ecd, which converts the witness into a low-norm vector in $\mathcal{R}_q^{tnk}$. The same transformation is applied to each entry of the query matrix given in the LPCP protocol, converting it into a matrix representation in $\mathcal{R}_q^{tnk \times 4}$. This transformation preserves the structure of the computation by leveraging the isomorphism between field and ring elements. To ensure the zero-knowledge property, the prover introduces an additional hiding vector $\vec{\mu}$ during the commitment phase. Since this vector is not part of the R1CS instance, a zero matrix must be appended to the query matrix to maintain consistency, resulting in the modified matrix $\mathbf{Q}_{\text{modified}}^T = [\mathbf{Q}^T | \mathbf{0}^{4 \times tnk}]$. Furthermore, since challenges are not encrypted in this construction, we need to make

rows of $\mathbf{Q}_{\text{modified}}^T$ uniform to preserve the zero-knowledge property as in LatticeDV-IPA protocol. We adopt the same technique we used for computing norm checking challenges (See step 4 to 9 of Figure 8). Similar to LatticeDV-IPA protocol, for each row of the modified query matrix to which the prover responds, the verifier must validate this response using an additional challenge matrix. Therefore, the matrix $\mathbf{Q}_{\text{verify}}$ is constructed as the verification matrix. The rest of the protocol is straightforward. The prover and verifier execute the DV-LatticeIPA protocol with four additional challenge vectors, placed in $\mathbf{Q}_{\text{chall}}$ and verified using $\mathbf{Q}_{\text{verify}}$. By applying Claim 4.1 to the prover's responses to $\mathbf{Q}_{\text{chall}}$, the verifier can compute the inner product of $\vec{q}_r$ with the witness, where $\vec{q}_r$ is the r th row of $\mathbf{Q}_{\text{modified}}$. The verifier also verify the response to these challenges by checking the following equality, which is resulted from Equation 9:

$$\begin{aligned} \vec{\pi}_{\mathbf{Q}_{\text{verify}}} &= \mathbf{Q}_{\text{verify}} \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix} \\ &= \begin{bmatrix} (\vec{z}_{\text{sec}}[2\lambda+2]+1)(\vec{\pi}_{\text{chall}}[1]) - \langle \vec{p}, \vec{c} \rangle \\ (\vec{z}_{\text{sec}}[2\lambda+3]+1)(\vec{\pi}_{\text{chall}}[1]) - \langle \vec{p}, \vec{c} \rangle \\ (\vec{z}_{\text{sec}}[2\lambda+4]+1)(\vec{\pi}_{\text{chall}}[1]) - \langle \vec{p}, \vec{c} \rangle \\ (\vec{z}_{\text{sec}}[2\lambda+5]+1)(\vec{\pi}_{\text{chall}}[1]) - \langle \vec{p}, \vec{c} \rangle \end{bmatrix} \end{aligned} \quad (15)$$

Next, the verifier computes the vector $\vec{a}$ from $\mathbf{Q}_{\text{chall}} \cdot \begin{bmatrix} \vec{b} \\ \vec{\mu} \end{bmatrix}$ and decodes it using Dcd algorithm to obtain the vector $\vec{a}$. The rest of the protocol is the same as verification step in LPCP.

Theorem 6.1. *The lattice-based DV-SNARK in Figure 8 satisfies completeness, knowledge soundness and zero-knowledge properties.*

Proof. The completeness of the protocol follows directly from the completeness of both the DV-LatticeIPA and LPCP protocols. Knowledge soundness is ensured by the extractability of the DV-LatticeIPA protocol in Theorem 5.2. Finally, combining the zero-knowledge properties of the DV-LatticeIPA and LPCP protocols establishes the zero-knowledge property of the overall construction. $\square$

VII. IMPLEMENTATION AND EVALUATION

The construction is implemented in C language² and compiled with clang 14.0.0. It builds on the publicly available code from [33], which provides trapdoor sampling routines over ring arithmetic.

Note that our work can only be compared with [23], as other constructions discussed in [22] are vulnerable to quantum attacks. The benchmark results in Figure 9 is obtained by running both implementations on the same machine, running Ubuntu 22.04.5 LTS with a 4-core Intel(R) Core(TM) i7-12700H processor, 8 GB of RAM, and a 60 GB hard disk.

Prior to implementation, we determine suitable parameter settings to ensure 128-bit security. For this purpose, we use the estimation tool developed by Albrecht et al. [34], which allows selecting parameters such that solving the underlying MSIS instance requires at least 2^{128} operations.

Table I shows the parameters we have used in our implementation, and their selected values. The parameter β_{open} is

²<https://github.com/crypolover1998/LatticeBased-Dv-zkSNARK>

Lattice-based DV-zkSNARK

$\text{Setup}_{\text{DVSNAK}}(1^\lambda) \rightarrow (\mathbf{ck}, \mathbf{crs}, \mathbf{vk})$

- 1) Run $\text{Setup}_{\text{CRS}}$ algorithm of Figure 6 and derive $\mathbf{ck}, \mathbf{crs}$ and $\mathbf{vk}$.
- 2) Encode each entry of Q_{LPCP}^T using $\text{Ecd}(\cdot)$ and output matrix Q_{LPCP}^T .
- 3) Set $Q_{\text{modified}}^T = [Q_{\text{LPCP}}^T | 0^{4 \times tnk}]$.
- 4) for $r = 1, 2, 3, 4$ do:
- 5) $\text{Ssample } \mathbf{z}_{\text{sec}}^{(2\lambda+4+r)}, \mathbf{z}_{\text{sec}}^{(2\lambda+r)}$
- 6) $\tilde{\mathbf{z}}_{\text{G, chall}}^{(r)} \leftarrow \text{ChallTrap}(\mathbf{ck}, \mathbf{td}, \tilde{\mathbf{p}}, \tilde{\mathbf{q}}_r)$ where $\tilde{\mathbf{q}}_r$ is the r th row of Q_{modified}^T .
- 7) $\tilde{\mathbf{q}}_{\text{chall}}^{(r)} \leftarrow \text{Gaussian2Uniform}(\tilde{\mathbf{u}}_{\text{chall}}^{(1)}, \mathbf{z}_{\text{sec}}^{(2\lambda+4+r)}, \tilde{\mathbf{z}}_{\text{G, chall}}^{(r)})$.
- 8) $\tilde{\mathbf{z}}_{\text{G, verify}}^{(r)} \leftarrow \text{ChallTrap}(\mathbf{ck}, \mathbf{td}, \tilde{\mathbf{p}}, \tilde{\mathbf{q}}_{\text{chall}}^{(r)})$ and $\tilde{\mathbf{q}}_{\text{verify}}^{(r)} \leftarrow \text{Gaussian2Uniform}(\tilde{\mathbf{u}}_{\text{chall}}^{(1)}, \mathbf{z}_{\text{sec}}^{(2\lambda+r)}, \tilde{\mathbf{z}}_{\text{G, verify}}^{(r)})$.
- 9) Set Matrices $Q_{\text{chall}}, Q_{\text{verify}}$ and update $\tilde{\mathbf{z}}_{\text{sec}}$ as follows:

$$Q_{\text{chall}} = \begin{bmatrix} \tilde{\mathbf{q}}_{\text{chall}}^{(1)} \\ \tilde{\mathbf{q}}_{\text{chall}}^{(2)} \\ \tilde{\mathbf{q}}_{\text{chall}}^{(3)} \\ \tilde{\mathbf{q}}_{\text{chall}}^{(4)} \end{bmatrix}, \quad Q_{\text{verify}} = \begin{bmatrix} \tilde{\mathbf{q}}_{\text{verify}}^{(1)} \\ \tilde{\mathbf{q}}_{\text{verify}}^{(2)} \\ \tilde{\mathbf{q}}_{\text{verify}}^{(3)} \\ \tilde{\mathbf{q}}_{\text{verify}}^{(4)} \end{bmatrix}, \quad \tilde{\mathbf{z}}_{\text{sec}} = \begin{bmatrix} \tilde{\mathbf{z}}_{\text{sec}}^{(1+2\lambda)} \\ \mathbf{z}_{\text{sec}}^{(1+2\lambda)} \\ \mathbf{z}_{\text{sec}}^{(2+2\lambda)} \\ \vdots \\ \mathbf{z}_{\text{sec}}^{(8+2\lambda)} \end{bmatrix}$$

- 10) Output $\mathbf{ck}, \mathbf{crs} = (\mathbf{U}_{\text{chall}}, \mathbf{U}_{\text{verify}}, Q_{\text{chall}}, Q_{\text{verify}}), \mathbf{vk} = (\tilde{\mathbf{z}}_{\text{sec}}, \tilde{\mathbf{p}})$.

$\mathcal{P}(\mathbf{ck}, \mathbf{crs}, \tilde{\mathbf{a}})$

- 1) For each entry in vector $\tilde{\mathbf{a}}$, run the Ecd algorithm and compute $\tilde{\mathbf{b}}$ from it.
- 2) Run $\text{TACS.Commit}(\mathbf{ck}, \tilde{\mathbf{b}})$ and derive $\tilde{\mathbf{c}}$.
- 3) Run the prover algorithm in Figure 6 and derive $\tilde{\pi}_{\text{chall}}, \tilde{\pi}_{\text{verify}}$.
- 4) Set $\tilde{\pi}_{Q_{\text{chall}}} := Q_{\text{chall}} \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\mu} \end{bmatrix}$ and $\tilde{\pi}_{Q_{\text{verify}}} = Q_{\text{verify}} \cdot \begin{bmatrix} \tilde{\mathbf{b}} \\ \tilde{\mu} \end{bmatrix}$.
- 5) Send $(\tilde{\mathbf{c}}, \tilde{\pi}_{\text{chall}}, \tilde{\pi}_{\text{verify}}, \tilde{\pi}_{Q_{\text{chall}}}, \tilde{\pi}_{Q_{\text{verify}}})$ to the verifier.

$\mathcal{V}(\mathbf{crs}, \mathbf{vk})$

- 1) Run the verifier algorithm in Figure 6. If it returns 0, return 0 otherwise go to step 3.
- 2) Return 0 if Equation 15 does not hold.
- 3) compute $\tilde{\mathbf{a}} = \tilde{\pi}_{Q_{\text{chall}}} + \begin{bmatrix} \langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle - \tilde{\mathbf{z}}_{\text{sec}}^{[2\lambda+6]} \tilde{\pi}_{\text{chall}}^{[1]} \\ \langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle - \tilde{\mathbf{z}}_{\text{sec}}^{[2\lambda+7]} \tilde{\pi}_{\text{chall}}^{[1]} \\ \langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle - \tilde{\mathbf{z}}_{\text{sec}}^{[2\lambda+8]} \tilde{\pi}_{\text{chall}}^{[1]} \\ \langle \tilde{\mathbf{p}}, \tilde{\mathbf{c}} \rangle - \tilde{\mathbf{z}}_{\text{sec}}^{[2\lambda+9]} \tilde{\pi}_{\text{chall}}^{[1]} \end{bmatrix}$ and set $\tilde{\mathbf{a}} = \text{Dcd}(\tilde{\mathbf{a}})$.
- 4) Run the $\mathcal{V}_{\text{LPCP}}$ and output 1 if it accpets. Otherwise output 0.

Fig. 8. Lattice-based designated verifier zkSNARK based on TACS.

TABLE I
LIST OF COMMON PARAMETERS AND THEIR DESCRIPTION.

Parameters	Description	Remarks	Value
q	ring modulus		$\approx 2^{32}$
d	ring dimension		16
n	dimension of trapdoors		2
k	number of trapdoors		
N_w	the witness length	$N_w = nk$	
β_{open}	witness l_∞ norm bound		2^{16}

chosen to ensure that the MSIS problem remains hard for the commitment matrices. Based on our estimation, setting $\beta_{\text{open}} = 2^{16}$ yields security exceeding 128 bits for witness lengths ranging from 2^{10} to 2^{22} . Due to the isomorphism between ring and field described in Section II-G, the value $(2\beta_{\text{open}} - 2)^r$, where $r|d$, corresponds to the effective field size. Consequently, the soundness of the underlying LPCP protocol is bounded by $\Omega(\frac{1}{\beta_{\text{open}}^r})$. By choosing $r = d$ the soundness of the LPCP protocol becomes approximately 2^{-256} . We set $n = 2$ for the trapdoor dimension, which yields trapdoor

matrices in $\mathcal{R}_q^{2 \times 66}$. To ensure security, the variance of the vectors $\tilde{\mathbf{p}}$ and $\tilde{\mathbf{z}}$ must be chosen sufficiently small, typically less than 2^{10} . This ensures that finding short pre-images for matrices of size 2×66 remains computationally hard, and only an entity possessing the corresponding trapdoor can efficiently compute such pre-images. The parameter k is then determined directly by fixing the witness length and the value of n .

TABLE II
PROOF AND CRS SIZE FOR $\lambda = 128$.

Metric	This Work	LUNA [23]
Proof Size (KB)	20.75	8.3
Full CRS Size (GB)	1.3	11

Table II presents a comparison of proof and CRS sizes. In our construction, the proof size includes both the commitment and the responses to the verifier's challenges size. Our CRS size is significantly smaller than that of LUNA. This improvement is primarily due to two factors: First, earlier constructions often rely on homomorphic encryption, whose

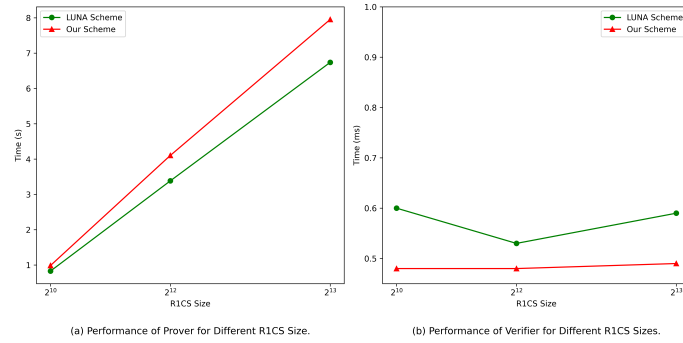


Fig. 9. Performance comparison of our scheme, and LUNA [23].

outputs are inherently large. Second, to achieve the desired level of soundness, these protocols typically require multiple repetitions, further increasing the CRS size. In contrast, our construction avoids these overheads.

We also compare our construction with LUNA in terms of execution time, as shown in Figure 9. Since LUNA is not a complete SNARK, we provide an approximate comparison based on analogous components. On the prover side, we exclude LPCP prover time for both schemes. For LUNA, we measure homomorphic addition and CRS expansion times. As CRS expansion is not implemented in LUNA, we implemented it using AES as a PRF to simulate seed-to-public-key expansion and incorporated this cost into the additive homomorphism time. In our scheme, the prover’s time includes only commitment generation and responses to verifier challenges. On the verifier side, we exclude LPCP verification and compare LUNA’s decryption time with our response verification time. Due to our hardware limits, we could simulate LUNA only for circuits up to size 2^{13} , restricting our comparison to this range. We simulate the prover and verifier with fewer number of challenges, then extrapolate the total execution time, noting that core operations scale linearly with the number of challenges. The main bottleneck in our protocol is the range proof, which requires 2λ challenge vectors and corresponding responses, which results in an increased overhead. However, the protocol remains succinct and practical.

VIII. CONCLUSIONS AND FUTURE WORKS

DV-zkSNARKs retain the broad applicability of traditional zkSNARKs while enabling verification restricted to a designated party. They have applications in areas such as verifiable computations and anonymous credentials. To the best of our knowledge, this work presents the first lattice-based DV-zkSNARK construction from the standard Module-SIS assumption. Prior lattice-based approaches typically rely on non-standard assumptions, such as linear targeted malleability, which are not well-established. Furthermore, these approaches ensure knowledge soundness only under non-adaptive conditions. In contrast, our scheme achieves adaptive knowledge soundness in the random oracle model by leveraging coordinate-wise special soundness. This approach to proving knowledge soundness is inherently confined to the random oracle model.

A key technical departure from prior works is that, unlike previous works that rely on homomorphic encryption, our construction employs a new inner-product argument combined with a modified Ajtai commitment scheme. The inner-product argument itself may be of independent interest.

Our implementation demonstrates that the prover and verifier achieve efficiency comparable to the best existing works, while the size of the CRS is reduced by a factor of 10. However, the commitment and proof sizes are approximately 2.5 times larger, primarily due to our reliance on standard assumptions. An important direction for future research is therefore to reduce these sizes, which would further improve the practicality of our scheme.

ACKNOWLEDGMENT

We have used ChatGPT for grammatical refinement and language editing. No part of the technical content, including proofs or constructions, was generated using ChatGPT.

REFERENCES

- [1] J. Kilian, “A note on efficient zero-knowledge proofs and arguments,” in *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*, 1992, pp. 723–732.
- [2] M. Dellepere, P. Mishra, and A. Shirzad, “Garuda and Pari: Faster and Smaller SNARKs via Equi-efficient Polynomial Commitments,” *Cryptology ePrint Archive*, 2024.
- [3] H. Lipmaa, “Polymath: Groth16 is not the limit,” in *Annual International Cryptology Conference*. Springer, 2024, pp. 170–206.
- [4] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part II 35*. Springer, 2016, pp. 305–326.
- [5] S. Setty, “Spartan: Efficient and general-purpose zkSNARKs without trusted setup,” in *Annual International Cryptology Conference*. Springer, 2020, pp. 704–737.
- [6] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, “Marlin: Preprocessing zkSNARKs with universal and updatable SRS,” in *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I 39*. Springer, 2020, pp. 738–768.
- [7] S. Atapoor, K. Bagheri, H. VL Pereira, and J. Spiessens, “Verifiable FHE via lattice-based SNARKs,” *IACR Communications in Cryptology (CiC)*, vol. 1, no. 1, 2024.
- [8] D. Boneh, Y. Ishai, A. Sahai, and D. J. Wu, “Lattice-based SNARKs and their application to more efficient obfuscation,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2017, pp. 247–277.

- [9] G. Arnon, J. Dujmovic, and Y. Ishai, “Designated-verifier SNARGs with one group element,” in *Annual International Cryptology Conference*. Springer, 2025, pp. 192–224.
- [10] M. R. Albrecht, V. Cini, R. W. Lai, G. Malavolta, and S. A. Thyagarajan, “Lattice-based SNARKs: publicly verifiable, preprocessing, and recursively composable,” in *Annual International Cryptology Conference*. Springer, 2022, pp. 102–132.
- [11] H. Wee and D. J. Wu, “Lattice-based functional commitments: Fast verification and cryptanalysis,” in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2023, pp. 201–235.
- [12] W. Beullens and G. Seiler, “LaBRADOR: compact proofs for R1CS from module-SIS,” in *Annual International Cryptology Conference*. Springer, 2023, pp. 518–548.
- [13] M. R. Albrecht, G. Fenzi, O. Lapiha, and N. K. Nguyen, “SLAP: succinct lattice-based polynomial commitments from standard assumptions,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2024, pp. 90–119.
- [14] I. Hwang, J. Seo, and Y. Song, “Concretely efficient lattice-based polynomial commitment from standard assumptions,” in *Annual International Cryptology Conference*. Springer, 2024, pp. 414–448.
- [15] N. K. Nguyen and G. Seiler, “GreyHound: fast polynomial commitments from lattices,” in *Annual International Cryptology Conference*. Springer, 2024, pp. 243–275.
- [16] N. Bitansky, A. Chiesa, Y. Ishai, O. Paneth, and R. Ostrovsky, “Succinct non-interactive arguments via linear interactive proofs,” in *Proceedings of the 10th theory of cryptography conference on Theory of Cryptography*, 2013, pp. 315–333.
- [17] R. Gennaro, M. Minelli, A. Nitulescu, and M. Orrù, “Lattice-based zk-SNARKs from square span programs,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 556–573.
- [18] Y. Ishai, H. Su, and D. J. Wu, “Shorter and faster post-quantum designated-verifier zkSNARKs from lattices,” in *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, 2021, pp. 212–234.
- [19] A. Nitulescu, “Lattice-based zero-knowledge SNARGs for arithmetic circuits,” in *Progress in Cryptology–LATINCRYPT 2019: 6th International Conference on Cryptology and Information Security in Latin America, Santiago de Chile, Chile, October 2–4, 2019, Proceedings 6*. Springer, 2019, pp. 217–236.
- [20] H. Chung, D. Kim, J. H. Kim, and J. Kim, “Amortized efficient zk-SNARK from linear-only RLWE encodings,” *Journal of Communications and Networks*, vol. 25, no. 3, pp. 271–284, 2023.
- [21] K. Naganuma, M. Yoshino, A. Inoue, Y. Matsuoka, M. Okazaki, and N. Kunihiro, “Post-Quantum zk-SNARK for arithmetic circuits using QAPs,” in *2020 15th Asia Joint Conference on Information Security (AsiaJCIS)*. IEEE, 2020, pp. 32–39.
- [22] T. Debris-Alazard, P. Fallahpour, and D. Stehlé, “Quantum Oblivious LWE Sampling and Insecurity of Standard Model Lattice-Based SNARKs,” in *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, 2024, pp. 423–434.
- [23] R. Steinfeld, A. Sakzad, M. F. Esgin, V. Kuchta, M. Yassi, and R. K. Zhao, “LUNA: Quasi-Optimally succinct Designated-Verifier Zero-Knowledge Arguments from Lattices,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 3167–3181.
- [24] M. Ajtai, “Generating hard instances of lattice problems,” in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, 1996, pp. 99–108.
- [25] C. Boschini, A. Takahashi, and M. Tibouchi, “MuSig-L: Lattice-based multi-signature with single-round online phase,” in *Annual International Cryptology Conference*. Springer, 2022, pp. 276–305.
- [26] A. Bishnoi, P. L. Clark, A. Potukuchi, and J. R. Schmitt, “On zeros of a polynomial in a finite grid,” *Combinatorics, Probability and Computing*, vol. 27, no. 3, pp. 310–333, 2018.
- [27] C. Baum and V. Lyubashevsky, “Simple amortized proofs of shortness for linear relations over polynomial rings,” *Cryptology ePrint Archive*, 2017.
- [28] V. Lyubashevsky, C. Peikert, and O. Regev, “A toolkit for ring-LWE cryptography,” in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2013, pp. 35–54.
- [29] G. Fenzi, H. Moghaddas, and N. K. Nguyen, “Lattice-based polynomial commitments: Towards asymptotic and concrete efficiency,” *Journal of Cryptology*, vol. 37, no. 3, p. 31, 2024.
- [30] D. Micciancio and C. Peikert, “Trapdoors for lattices: Simpler, tighter, faster, smaller,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2012, pp. 700–718.
- [31] V. Lyubashevsky, N. K. Nguyen, and G. Seiler, “Shorter lattice-based zero-knowledge proofs via one-time commitments,” in *IACR International Conference on Public-Key Cryptography*. Springer, 2021, pp. 215–241.
- [32] C. Baum, J. Bootle, A. Cerulli, R. Del Pino, J. Groth, and V. Lyubashevsky, “Sub-linear lattice-based zero-knowledge arguments for arithmetic circuits,” in *Annual International Cryptology Conference*. Springer, 2018, pp. 669–699.
- [33] P. Bert, G. Eberhart, L. Prabel, A. Roux-Langlois, and M. Sabt, “Implementation of lattice trapdoors on modules and applications,” in *Post-Quantum Cryptography: 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20–22, 2021, Proceedings 12*. Springer, 2021, pp. 195–214.
- [34] M. R. Albrecht, R. Player, and S. Scott, “On the concrete hardness of learning with errors,” *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, 2015.

APPENDIX

ADDITIONAL PRILIMINARIES

Definition 1.1 (Gadget Matrix). Let $b \geq 2$. We define $\delta := \lceil \log_b q \rceil + 1$, and set $\vec{g}^\top = [1, b, \dots, b^{\delta-1}] \in \mathbb{R}_q^{1 \times \delta}$, and the gadget matrix $\mathbf{G}_n := \mathbf{I}_n \otimes \vec{g}^\top \in \mathbb{R}_q^{n \times (n \cdot \delta)}$.

Definition 1.2 (G-Trapdoor). Let $\mathbf{A} \in \mathbb{R}_q^{n \times m}$, $\mathbf{H} \in \mathbb{R}_q^{n \times m}$, and $\mathbf{G} \in \mathbb{R}_q^{n \times n\delta}$ be gadget matrix. A **G-trapdoor** for $\mathbf{A}$ is a matrix $\mathbf{R} \in \mathbb{R}_q^{m \times n\delta}$ with $\mathbf{A}\mathbf{R} = \mathbf{G}$.

Lemma 1.1 (Trapdoor Generation). [30] Let $\mathcal{D}$ be a discrete Gaussian distribution over $\mathbb{R}_q^{(n) \times (n\delta)}$ with parameter s , and $\mathbf{G}_n$ be the gadget matrix. Take $m = n\delta + n$. Then, there exists a PPT algorithm $\text{TrapGen}(n, m, \mathcal{D})$ that, with overwhelming probability, returns two matrices $(\mathbf{A}, \mathbf{R}) \in \mathbb{R}_q^{n \times m} \times \mathbb{R}_q^{m \times n\delta}$ such that $\mathbf{A} \begin{bmatrix} \mathbf{R} \\ \mathbf{I} \end{bmatrix} = \mathbf{G}_n \pmod{q}$. Moreover, $\mathbf{A}$ is statistically close to a uniformly random matrix.

TrapGen [30]

$\text{TrapGen}(n, m, \mathcal{D}) \rightarrow \mathbf{A}, \mathbf{R}$

- 1) Sample $\mathbf{B} \leftarrow \mathcal{U}(\mathbb{R}_q^{n \times n})$ and $\mathbf{R} \leftarrow \mathcal{D}_s^{n \times n\delta}$.
- 2) Output $\mathbf{A} = [\mathbf{B} | \mathbf{G} - \mathbf{B}\mathbf{R}]$ and trapdoor $\mathbf{R}$.

Fig. 10. The algorithm for sampling a uniform matrix along with its trapdoor.

Lemma 1.2 (Preimage Sampling). [30] There exists a PPT algorithm $\text{SamplePre}(\mathbf{A}, \mathbf{R}, \vec{v}, \sigma)$ that takes as input a matrix $\mathbf{A}$, $\mathbf{R}$, a target vector $\vec{u}$, and a Gaussian parameter σ , and outputs a vector $\vec{u}$ such that $\mathbf{A} \cdot \vec{u} \equiv \vec{v} \pmod{q}$.

SamplePre [30]

$\text{SamplePre}(\mathbf{A}, \mathbf{R}, \vec{v}, \sigma_u) \rightarrow \vec{u}$

- 1) Sample fresh perturbation vector $\vec{p}$ from $\mathcal{D}_{\mathcal{R}^m}$ with covariance matrix $\sigma_u^2 \mathbf{I} - s^2 \begin{bmatrix} \mathbf{R} \\ \mathbf{I} \end{bmatrix} \mathbf{I} \begin{bmatrix} \mathbf{R}^\top & \mathbf{I}^\top \end{bmatrix}$.
- 2) Sample spherical $\vec{z} \leftarrow \mathcal{D}_s^m$ such that $\mathbf{G}\vec{z} = \vec{u} - \mathbf{A}\vec{p}$.
- 3) Output $\vec{u} = \vec{p} + \begin{bmatrix} \mathbf{R} \\ \mathbf{I} \end{bmatrix} \vec{z}$.

Fig. 11. The algorithm for sampling pre-images.